



Вместо введения



Вместо введения

Задача

Необходимо перемножить две матрицы размерностью $N \times N$

Решение для $N=2$

$$C_{ij} = \sum_{s=1}^n A_{is} B_{sj} \quad A = \begin{pmatrix} a_1 & b_1 \\ a_2 & b_2 \end{pmatrix} \quad B = \begin{pmatrix} c_1 & d_1 \\ c_2 & d_2 \end{pmatrix}$$

$$\begin{pmatrix} a_1 & b_1 \\ a_2 & b_2 \end{pmatrix} \begin{pmatrix} c_1 & d_1 \\ c_2 & d_2 \end{pmatrix} = \begin{pmatrix} ? & ? \\ ? & ? \end{pmatrix}$$



Вместо введения

Задача

Необходимо перемножить две матрицы размерностью $N \times N$

Решение

$$C_{ij} = A_{i1}B_{1j} + A_{i2}B_{2j}$$
$$C_{11} = A_{11}B_{11} + A_{12}B_{21}$$
$$\begin{pmatrix} a_1 & b_1 \\ a_2 & b_2 \end{pmatrix} \begin{pmatrix} c_1 & d_1 \\ c_2 & d_2 \end{pmatrix} = \begin{pmatrix} a_1c_1 + b_1c_2 & ? \\ ? & ? \end{pmatrix}$$

$$\begin{pmatrix} a_1 & b_1 \\ a_2 & b_2 \end{pmatrix} \begin{pmatrix} c_1 & d_1 \\ c_2 & d_2 \end{pmatrix} = \begin{pmatrix} a_1c_1 + b_1c_2 & a_1d_1 + b_1d_2 \\ a_2c_1 + b_2c_2 & a_2d_1 + b_2d_2 \end{pmatrix}$$



Вместо введения

Задача

Необходимо перемножить две матрицы размерностью $N \times N$

Решение

$$C_{ij} = \sum_{s=1}^n A_{is} B_{sj} \quad \begin{pmatrix} A_{11} & * & A_{1n} \\ * & * & * \\ A_{n1} & * & A_{nn} \end{pmatrix} \begin{pmatrix} B_{11} & * & B_{1n} \\ * & * & * \\ B_{n1} & * & B_{nn} \end{pmatrix} = \begin{pmatrix} ? & * & ? \\ * & * & * \\ ? & * & ? \end{pmatrix}$$

$$C_{ij} = A_{i1} B_{1j} + A_{i2} B_{2j} + \dots + A_{in} B_{nj}$$

$$\begin{pmatrix} A_{11} & * & A_{1n} \\ * & * & * \\ A_{n1} & * & A_{nn} \end{pmatrix} \begin{pmatrix} B_{11} & * & B_{1n} \\ * & * & * \\ B_{n1} & * & B_{nn} \end{pmatrix} = \begin{pmatrix} ? & * & ? \\ * & * & * \\ ? & * & ? \end{pmatrix}$$



Вместо введения

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        for (int k = 0; k < n; k++)
            C[i][j] = C[i][j] + A[i][k] * B[k][j];
```

$i=0, j=0, k=0$ $C[0][0] = C[0][0] + A[0][0] * B[0][0];$

$i=0, j=0, k=1$ $C[0][0] = C[0][0] + A[0][1] * B[1][0];$

$i=0, j=0, k=n-1$ $C[0][0] = C[0][0] + A[0][n-1] * B[n-1][0];$

$$\begin{pmatrix} \boxed{A_{00} * A_{0(n-1)}} \\ * \\ A_{(n-1)0} * A_{(n-1)(n-1)} \end{pmatrix} \begin{pmatrix} \boxed{B_{00}} * B_{0(n-1)} \\ * \\ B_{(n-1)0} * B_{(n-1)(n-1)} \end{pmatrix} = \begin{pmatrix} \boxed{A_{00}B_{00} + \dots + A_{(n-1)0}B_{0(n-1)}} * * \\ * * * \\ ? * ? \end{pmatrix}$$



Вместо введения

```
for (int i = 0; i < n; i++)
    for (int k = 0; k < n; k++)
        for (int j = 0; j < n; j++)
            C[i][j] = C[i][j] + A[i][k] * B[k][j];
```

i=0, k=0, j=0

C[0][0] = C[0][0] + A[0][0] * B[0][0];

i=0, k=0, j=1

C[0][1] = C[0][1] + A[0][0] * B[0][1];

i=0, k=0, j=n-1

C[0][n] = C[0][n-1] + A[0][0] * B[0][n-1];

$$\begin{pmatrix} \boxed{A_{00}} & * & A_{0(n-1)} \\ * & * & * \\ A_{(n-1)0} & * & A_{(n-1)(n-1)} \end{pmatrix} \begin{pmatrix} \boxed{B_{00}} & * & \boxed{B_{0(n-1)}} \\ * & * & * \\ B_{(n-1)0} & * & B_{(n-1)(n-1)} \end{pmatrix} = \begin{pmatrix} \boxed{A_{00}B_{00}} & * & \boxed{A_{00}B_{0(n-1)}} \\ * & * & * \\ ? & * & ? \end{pmatrix}$$



Вместо введения

```
for (int i = 0; i < n; i++)  
    for (int k = 0; k < n; k++)  
        for (int j = 0; j < n; j++)  
            C[i][j] = C[i][j] + A[i][k] * B[k][j];
```

i=0, k=1, j=0 C[0][0] = C[0][0] + A[0][1] * B[1][0];

i=0, k=1, j=1 C[0][1] = C[0][1] + A[0][1] * B[1][1];

i=0, k=1, j=n-1 C[0][n] = C[0][n-1] + A[0][1] * B[1][n-1];

$$\begin{pmatrix} A_{00} & * & A_{0(n-1)} \\ * & * & * \\ A_{(n-1)0} & * & A_{(n-1)(n-1)} \end{pmatrix} \begin{pmatrix} B_{00} & * & B_{0(n-1)} \\ * & * & * \\ B_{(n-1)0} & * & B_{(n-1)(n-1)} \end{pmatrix} = \begin{pmatrix} A_{00}B_{00} + A_{01}B_{10} & * & A_{00}B_{0(n-1)} + A_{01}B_{1(n-1)} \\ * & * & * \\ ? & * & ? \end{pmatrix}$$



Вместо введения

```
for (int j = 0; j < n; j++)
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            C[i][j] = C[i][j] + A[i][k] * B[k][j];
```

j=0, k=0, i=0

C[0][0] = C[0][0] + A[0][0] * B[0][0];

j=0, k=0, i=1

C[1][0] = C[1][0] + A[1][0] * B[0][0];

j=0, k=0, i=n-1

C[n-1][0] = C[n-1][0] + A[n-1][0] * B[0][0];

$$\begin{pmatrix} A_{00} & * & A_{0(n-1)} \\ * & * & * \\ A_{(n-1)0} & * & A_{(n-1)(n-1)} \end{pmatrix} \begin{pmatrix} B_{00} & * & B_{0(n-1)} \\ * & * & * \\ B_{(n-1)0} & * & B_{(n-1)(n-1)} \end{pmatrix} = \begin{pmatrix} A_{00}B_{00} & * & ? \\ * & * & * \\ A_{(n-1)0}B_{00} & * & ? \end{pmatrix}$$



Вместо введения

```
for (int j = 0; j < n; j++)  
    for (int k = 0; k < n; k++)  
        for (int i = 0; i < n; i++)  
            C[i][j] = C[i][j] + A[i][k] * B[k][j];
```

j=0, k=1, i=0 C[0][0] = C[0][0] + A[0][1] * B[1][0];

j=0, k=1, i=1 C[1][0] = C[1][0] + A[1][1] * B[1][0];

j=0, k=1, i=n-1 C[n-1][0] = C[n-1][0] + A[n-1][1] * B[1][0];

$$\begin{pmatrix} A_{00} & * & A_{0(n-1)} \\ * & * & * \\ A_{(n-1)0} & * & A_{(n-1)(n-1)} \end{pmatrix} \begin{pmatrix} B_{00} & * & B_{0(n-1)} \\ * & * & * \\ B_{(n-1)0} & * & B_{(n-1)(n-1)} \end{pmatrix} = \begin{pmatrix} A_{00}B_{00} + A_{01}B_{10} & * & ? \\ * & * & * \\ A_{(n-1)0}B_{00} + A_{(n-1)1}B_{10} & * & ? \end{pmatrix}$$



Вместо введения

А есть ли разница?

```
for (int j = 0; j < n; j++)
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            C[i][j] = C[i][j] + A[i][k] * B[k][j];
```

j=0, k=1, i=0 C[0][0] = C[0][0] + A[0][1] * B[1][0];

j=0, k=1, i=1 C[1][0] = C[1][0] + A[1][1] * B[1][0];

j=0, k=1, i=n-1 C[n-1][0] = C[n-1][0] + A[n-1][1] * B[1][0];

$$\begin{pmatrix} A_{00} & * & A_{0(n-1)} \\ * & * & * \\ A_{(n-1)0} & * & A_{(n-1)(n-1)} \end{pmatrix} \begin{pmatrix} B_{00} & * & B_{0(n-1)} \\ * & * & * \\ B_{(n-1)0} & * & B_{(n-1)(n-1)} \end{pmatrix} = \begin{pmatrix} A_{00}B_{00} + A_{01}B_{10} & * & ? \\ * & * & * \\ A_{(n-1)0}B_{00} + A_{(n-1)1}B_{10} & * & ? \end{pmatrix}$$



Вместо введения

Хранение массива в памяти

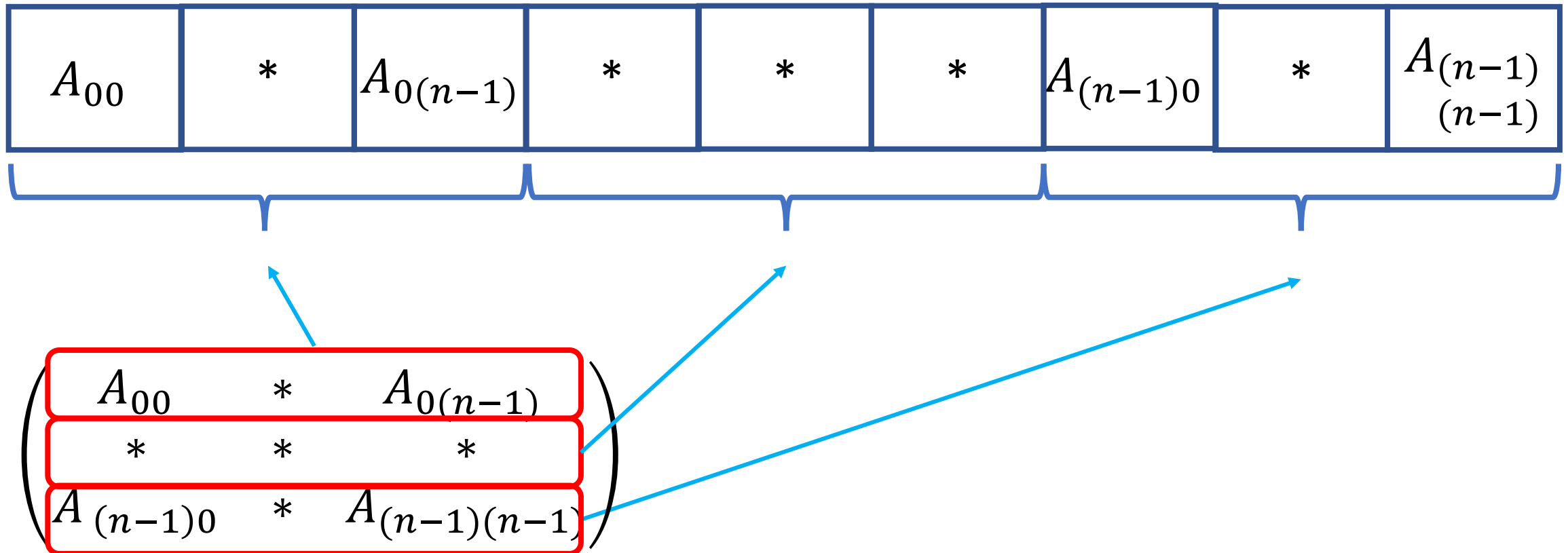
A_{00}	*	$A_{0(n-1)}$	*	*	*	$A_{(n-1)0}$	*	$A_{(n-1)(n-1)}$
----------	---	--------------	---	---	---	--------------	---	------------------

$$\begin{pmatrix} A_{00} & * & A_{0(n-1)} \\ * & * & * \\ A_{(n-1)0} & * & A_{(n-1)(n-1)} \end{pmatrix}$$



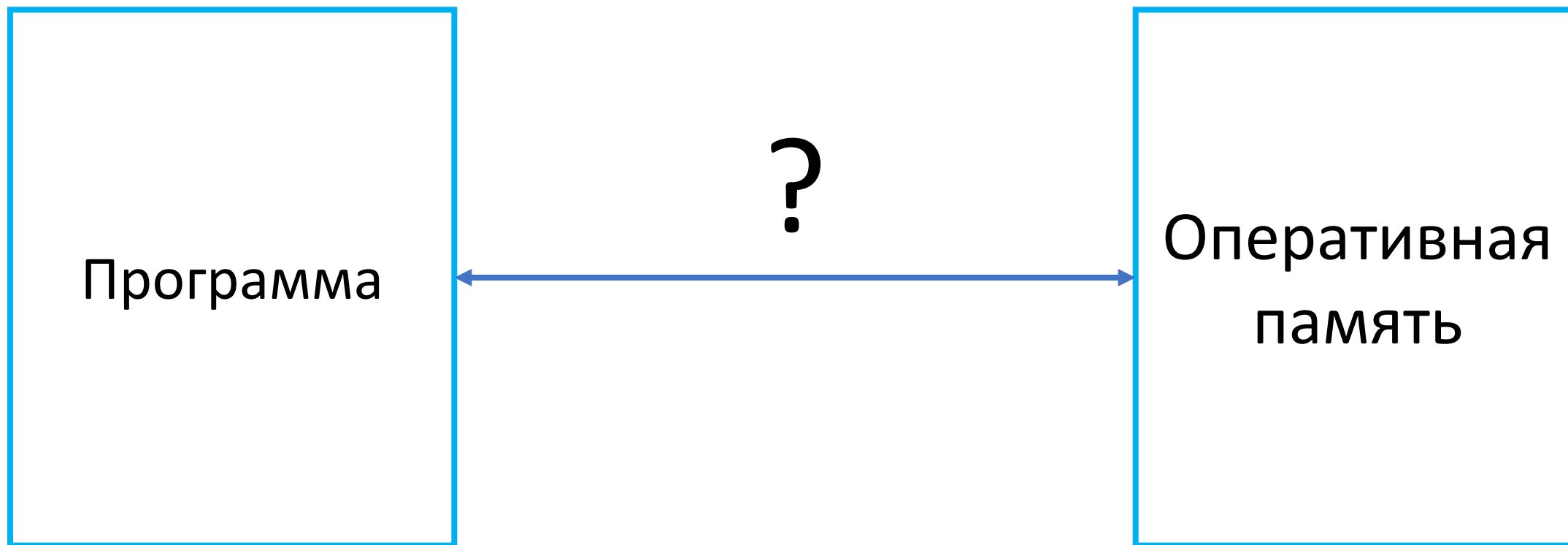
Вместо введения

Хранение массива в памяти



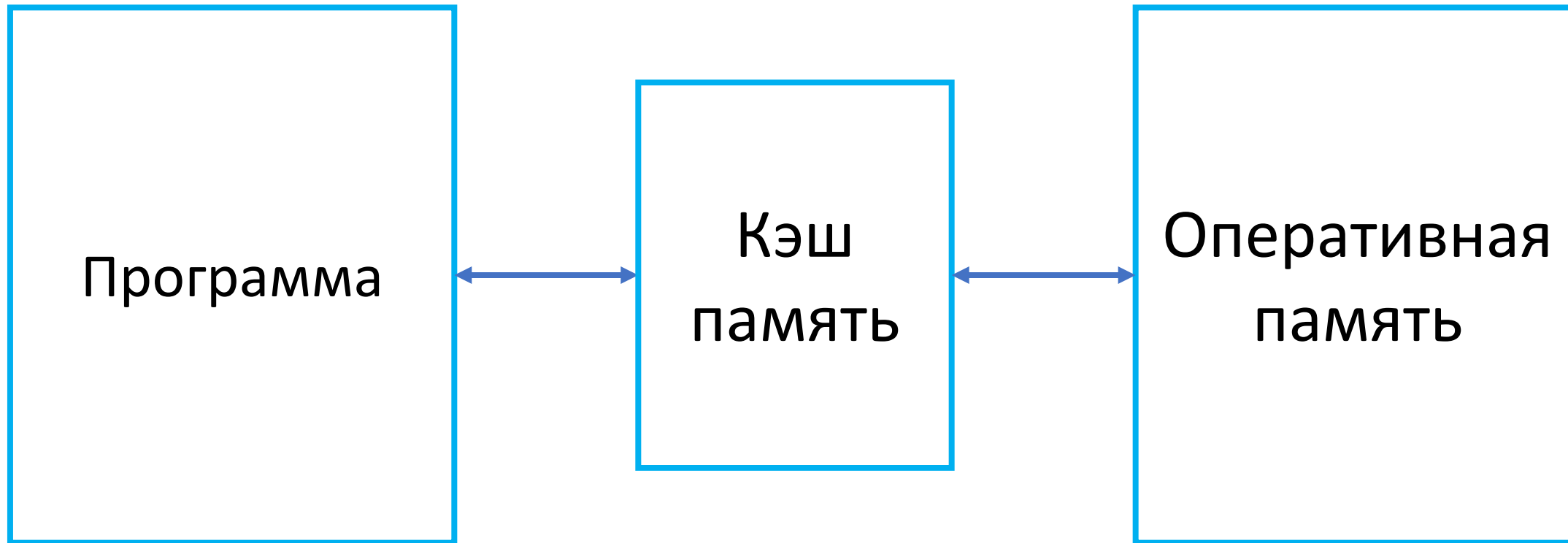


Вместо введения



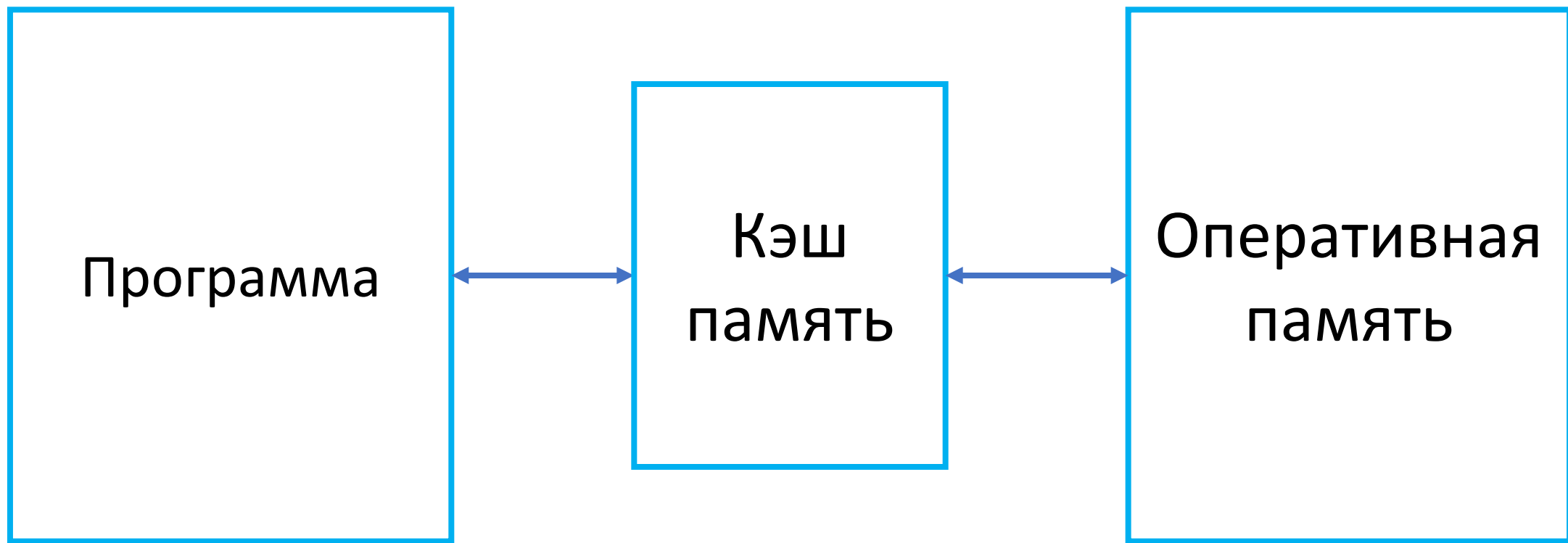


Вместо введения





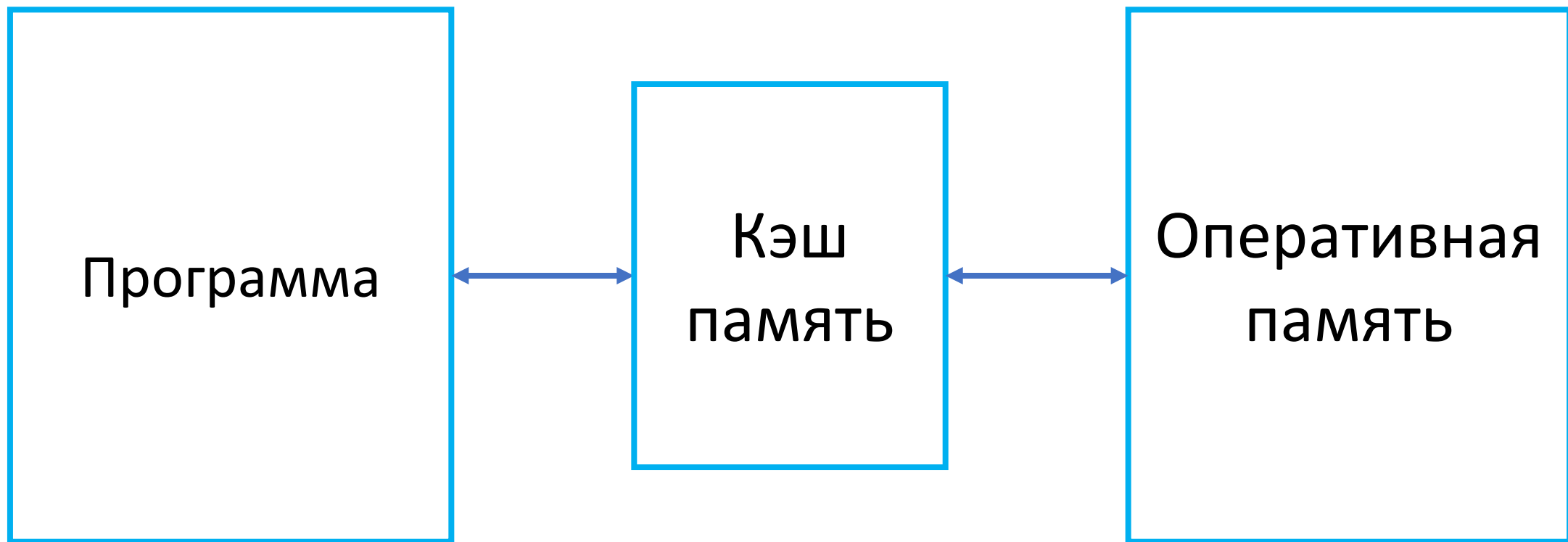
Вместо введения



1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---



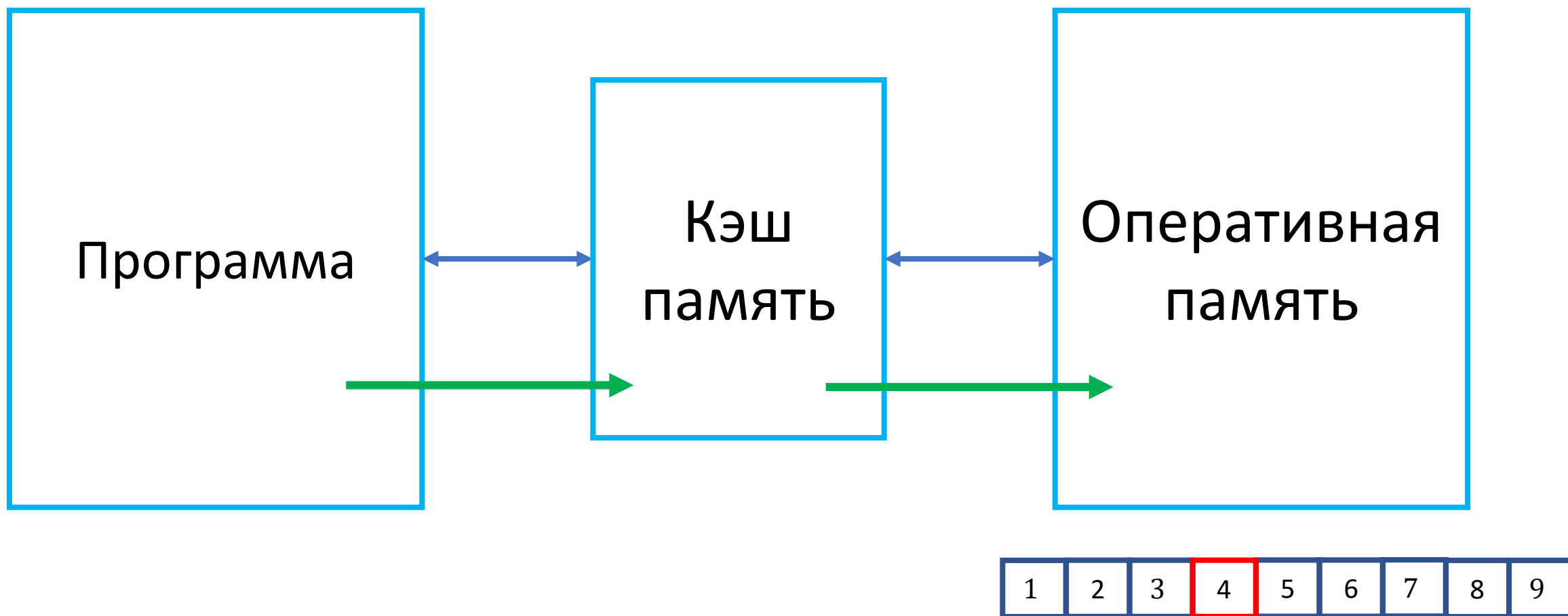
Вместо введения



1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

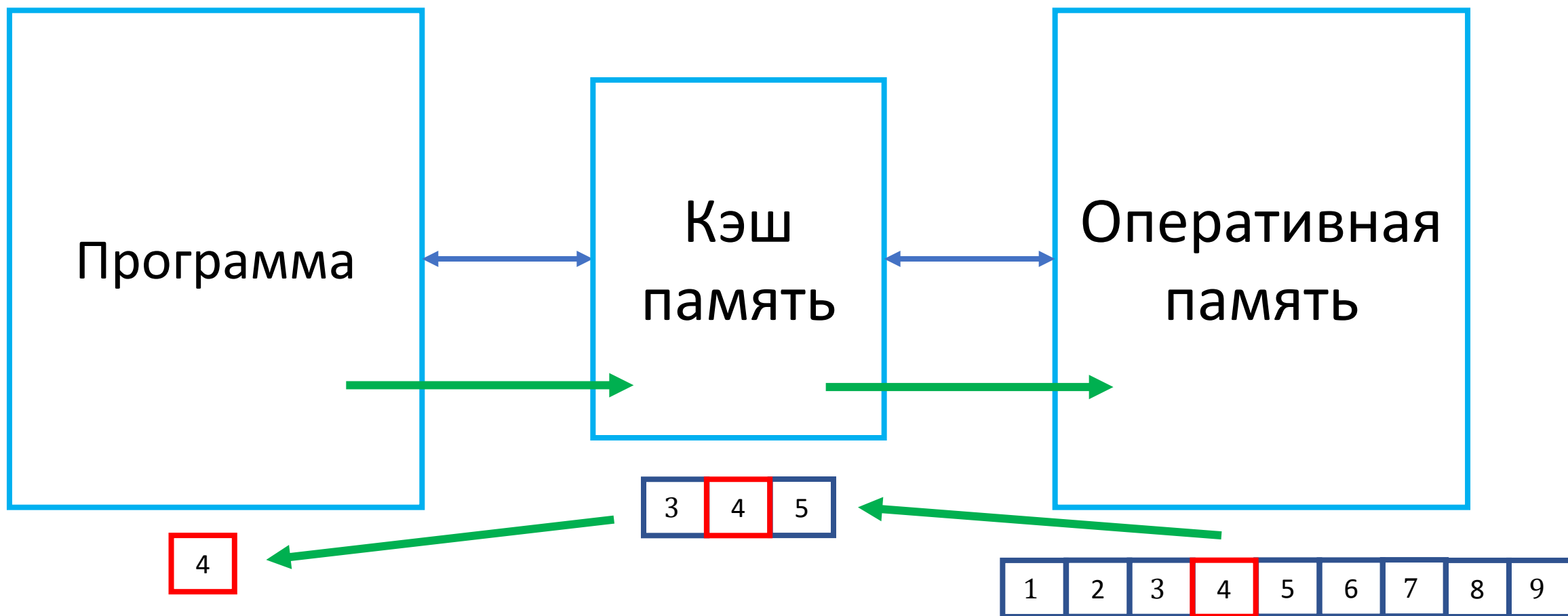


Вместо введения



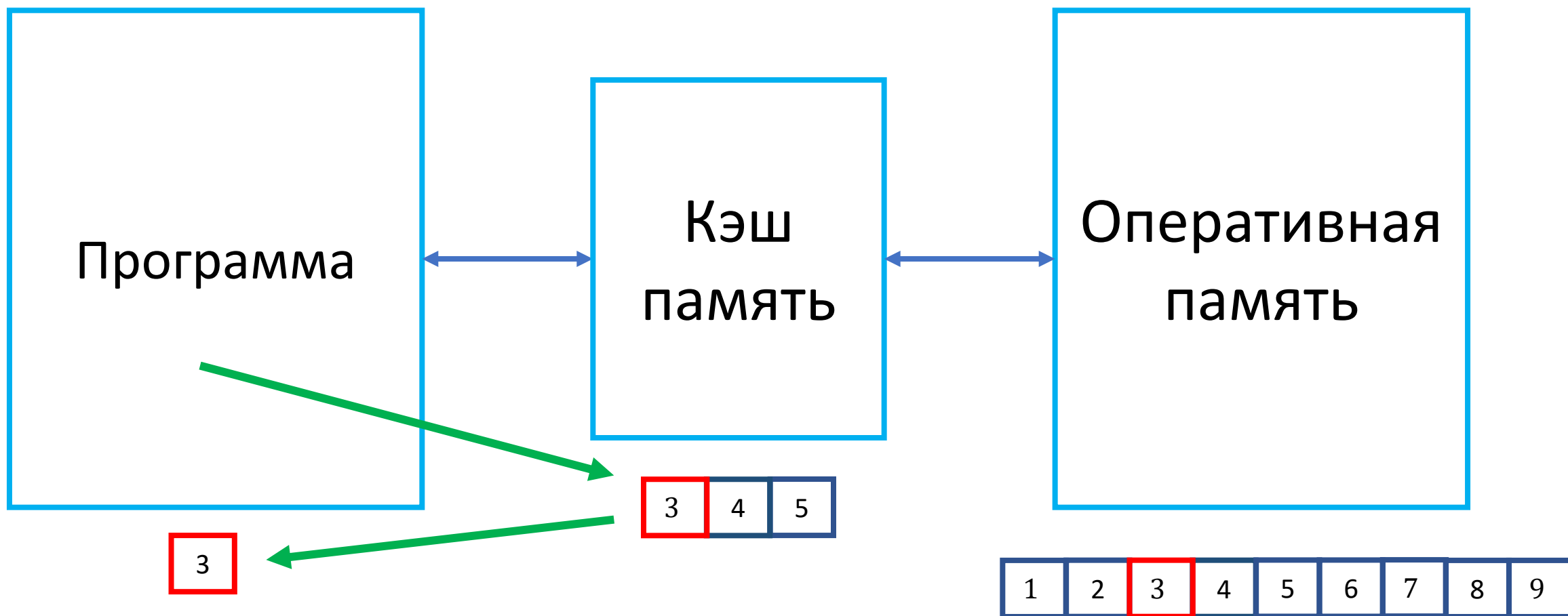


Вместо введения





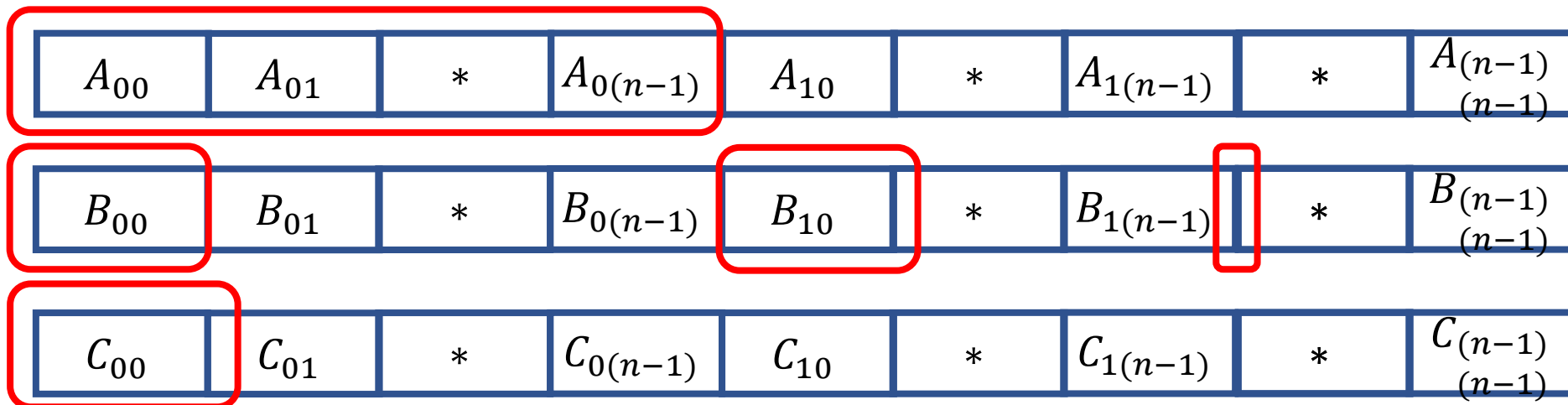
Вместо введения





Вместо введения

Оперативная память

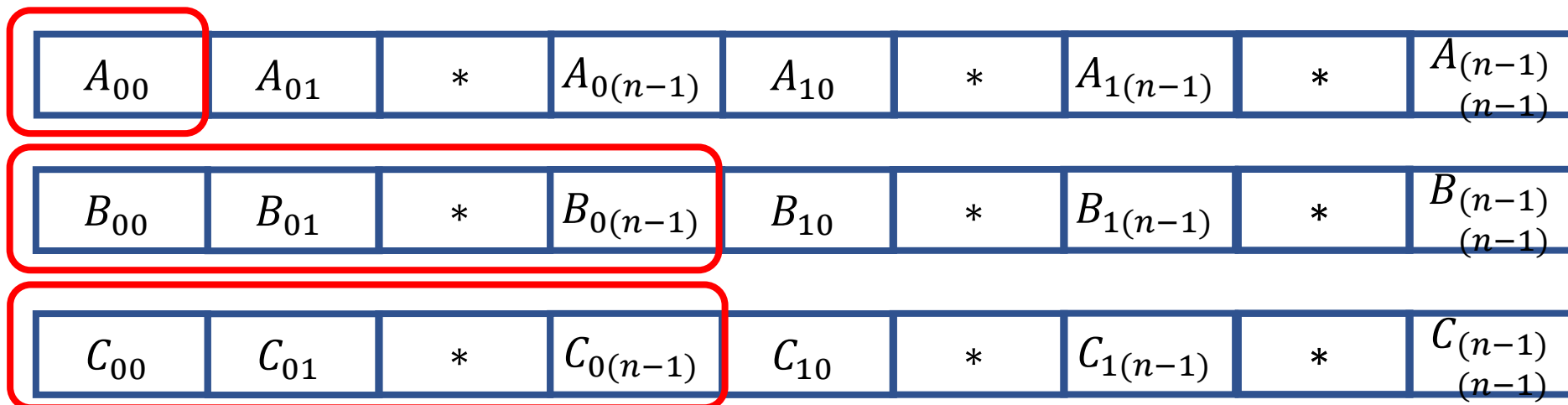


$$\begin{pmatrix} A_{00} & * & A_{0(n-1)} \\ * & * & * \\ A_{(n-1)0} & * & A_{(n-1)(n-1)} \end{pmatrix} \begin{pmatrix} B_{00} & * & B_{0(n-1)} \\ * & * & * \\ B_{(n-1)0} & * & B_{(n-1)(n-1)} \end{pmatrix} = \begin{pmatrix} A_{00}B_{00} + \dots + A_{(n-1)0}B_{0(n-1)} & * & * \\ * & * & * \\ ? & * & ? \end{pmatrix}$$



Вместо введения

Оперативная память

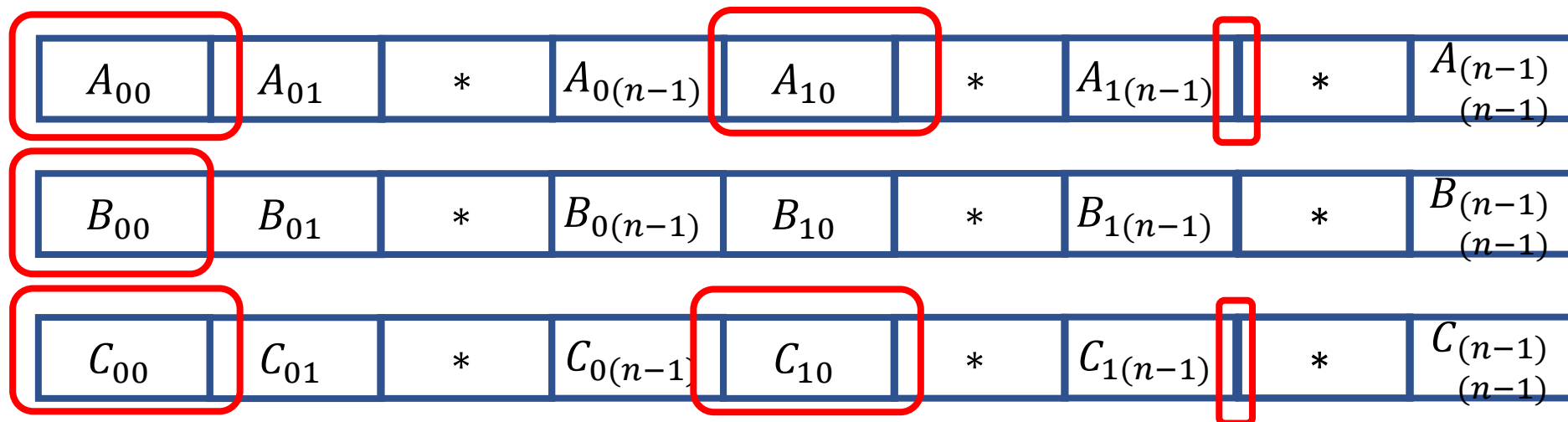


$$\begin{pmatrix} A_{00} & * & A_{0(n-1)} \\ * & * & * \\ A_{(n-1)0} & * & A_{(n-1)(n-1)} \end{pmatrix} \begin{pmatrix} B_{00} & * & B_{0(n-1)} \\ * & * & * \\ B_{(n-1)0} & * & B_{(n-1)(n-1)} \end{pmatrix} = \begin{pmatrix} A_{00}B_{00} & * & A_{00}B_{0(n-1)} \\ * & * & * \\ ? & * & ? \end{pmatrix}$$



Вместо введения

Оперативная память



$$\begin{pmatrix} A_{00} & * & A_{0(n-1)} \\ * & * & * \\ A_{(n-1)0} & * & A_{(n-1)(n-1)} \end{pmatrix} \begin{pmatrix} B_{00} & * & B_{0(n-1)} \\ * & * & * \\ B_{(n-1)0} & * & B_{(n-1)(n-1)} \end{pmatrix} = \begin{pmatrix} A_{00}B_{00} & * & ? \\ * & * & * \\ A_{(n-1)0}B_{00} & * & ? \end{pmatrix}$$



Вместо введения

$$1. \begin{pmatrix} \boxed{A_{00}} & * & \boxed{A_{0(n-1)}} \\ * & * & * \\ A_{(n-1)0} & * & A_{(n-1)(n-1)} \end{pmatrix} \begin{pmatrix} \boxed{B_{00}} & * & B_{0(n-1)} \\ * & * & * \\ B_{(n-1)0} & * & B_{(n-1)(n-1)} \end{pmatrix} = \begin{pmatrix} \boxed{A_{00}B_{00} + \dots + A_{(n-1)0}B_{0(n-1)}} & * & * \\ * & * & * \\ ? & * & ? \end{pmatrix} \quad \textbf{ijk}$$

$$2. \begin{pmatrix} \boxed{A_{00}} & * & A_{0(n-1)} \\ * & * & * \\ A_{(n-1)0} & * & A_{(n-1)(n-1)} \end{pmatrix} \begin{pmatrix} \boxed{B_{00}} & * & \boxed{B_{0(n-1)}} \\ * & * & * \\ B_{(n-1)0} & * & B_{(n-1)(n-1)} \end{pmatrix} = \begin{pmatrix} \boxed{A_{00}B_{00}} & * & \boxed{A_{00}B_{0(n-1)}} \\ * & * & * \\ ? & * & ? \end{pmatrix} \quad \textbf{ikj}$$

$$3. \begin{pmatrix} \boxed{A_{00}} & * & A_{0(n-1)} \\ * & * & * \\ \boxed{A_{(n-1)0}} & * & A_{(n-1)(n-1)} \end{pmatrix} \begin{pmatrix} \boxed{B_{00}} & * & B_{0(n-1)} \\ * & * & * \\ B_{(n-1)0} & * & B_{(n-1)(n-1)} \end{pmatrix} = \begin{pmatrix} \boxed{A_{00}B_{00}} & * & ? \\ * & * & * \\ \boxed{A_{(n-1)0}B_{00}} & * & ? \end{pmatrix} \quad \textbf{jki}$$



Профилирование

```
-----  
I1 cache:      32768 B, 64 B, 8-way associative  
D1 cache:      49152 B, 64 B, 12-way associative  
LL cache:      12582912 B, 64 B, 12-way associative  
Command:       ./MatrixMult 1000  
Data file:     cachegrind.out.298  
Events recorded: Ir I1mr I1mr Dr D1mr DLmr Dw D1mw DLmw  
Events shown:  Ir I1mr I1mr Dr D1mr DLmr Dw D1mw DLmw  
Event sort order: Ir I1mr I1mr Dr D1mr DLmr Dw D1mw DLmw  
Thresholds:    0.1 100 100 100 100 100 100 100 100  
Include dirs:  
User annotated:  
Auto-annotation: off  
-----
```

```
-----  
Ir          I1mr  I1mr  Dr          D1mr          DLmr          Dw          D1mw  DLmw  
-----  
204,295,449,017 1,353 1,340 88,100,459,991 4,678,273,607 5,479,281 4,024,205,192 755,691 754,578  PROGRAM TOTALS  
-----
```

```
-----  
Ir          I1mr  I1mr  Dr          D1mr          DLmr          Dw          D1mw  DLmw  file:function  
-----  
51,011,011,017 5      5 22,005,005,004 2,253,119,003 2,336,192 1,001,001,006      0      0  ??? :mult_jki  
51,011,011,017 5      5 22,005,005,004 1,142,599,004 2,388,068 1,001,001,006      0      0  ??? :mult_jik  
51,011,011,017 4      4 22,005,005,004 1,156,917,909 375,142 1,001,001,006      0      0  ??? :mult_ijk  
51,011,011,017 4      4 22,005,005,004 125,630,758 375,110 1,001,001,006      0      0  ??? :mult_ikj  
-----
```



Вместо введения

Вывод!

Кроме понимания алгоритмов нужно знать то «железо», под которое пишешь программу



Почему мы учим именно язык «С»

Jan 2021	Jan 2020	Change	Programming Language	Ratings	Change
1	2	⬆	C	17.38%	+1.61%
2	1	⬇	Java	11.96%	-4.93%
3	3		Python	11.72%	+2.01%
4	4		C++	7.56%	+1.99%
5	5		C#	3.95%	-1.40%
6	6		Visual Basic	3.84%	-1.44%
7	7		JavaScript	2.20%	-0.25%
8	8		PHP	1.99%	-0.41%
9	18	⬆	R	1.90%	+1.10%
10	23	⬆	Groovy	1.84%	+1.23%



Почему мы учим именно язык «С»

Jan 2022	Jan 2021	Change	Programming Language		Ratings	Change
1	3	↑		Python	13.58%	+1.86%
2	1	↓		C	12.44%	-4.94%
3	2	↓		Java	10.66%	-1.30%
4	4			C++	8.29%	+0.73%
5	5			C#	5.68%	+1.73%
6	6			Visual Basic	4.74%	+0.90%
7	7			JavaScript	2.09%	-0.11%
8	11	↑		Assembly language	1.85%	+0.21%
9	12	↑		SQL	1.80%	+0.19%
10	13	↑		Swift	1.41%	-0.02%



Почему мы учим именно язык «С»

Feb 2023	Feb 2022	Change	Programming Language		Ratings	Change
1	1			Python	15.49%	+0.16%
2	2			C	15.39%	+1.31%
3	4	▲		C++	13.94%	+5.93%
4	3	▼		Java	13.21%	+1.07%
5	5			C#	6.38%	+1.01%
6	6			Visual Basic	4.14%	-1.09%
7	7			JavaScript	2.52%	+0.70%
8	10	▲		SQL	2.12%	+0.58%
9	9			Assembly language	1.38%	-0.21%



Почему мы учим именно язык «С»

Feb 2024	Feb 2023	Change	Programming Language		Ratings	Change
1	1			Python	15.16%	-0.32%
2	2			C	10.97%	-4.41%
3	3			C++	10.53%	-3.40%
4	4			Java	8.88%	-4.33%
5	5			C#	7.53%	+1.15%
6	7	^		JavaScript	3.17%	+0.64%
7	8	^		SQL	1.82%	-0.30%
8	11	^		Go	1.73%	+0.61%
9	6	v		Visual Basic	1.52%	-2.62%



Почему мы учим именно язык «С»

Jan 2025	Jan 2024	Change	Programming Language		Ratings	Change
1	1			Python	23.28%	+9.32%
2	3	^		C++	10.29%	+0.33%
3	4	^		Java	10.15%	+2.28%
4	2	v		C	8.86%	-2.59%
5	5			C#	4.45%	-2.71%
6	6			JavaScript	4.20%	+1.43%
7	11	^^		Go	2.61%	+1.24%
8	9	^		SQL	2.41%	+0.95%
9	8	v		Visual Basic	2.37%	+0.77%



Статистика по github (2023)

Language	Primary Language for % of Repositories		Occur in % of Repositories		Total Number of Bytes	
Python		23.21%		31.94%		1.10 GB
TypeScript		13.19%		18.83%		0.19 GB
JavaScript		7.50%		29.60%		0.35 GB
Rust		4.19%		5.52%		63.21 MB
Go		4.02%		5.17%		0.18 GB
C#		4.02%		5.05%		0.31 GB
C++		3.84%		8.61%		1.14 GB
Jupyter Notebook		3.24%		5.14%		1.07 GB
Java		2.88%		4.30%		0.48 GB
Kotlin		2.71%		4.38%		60.23 MB
C		2.63%		8.37%		3.80 GB



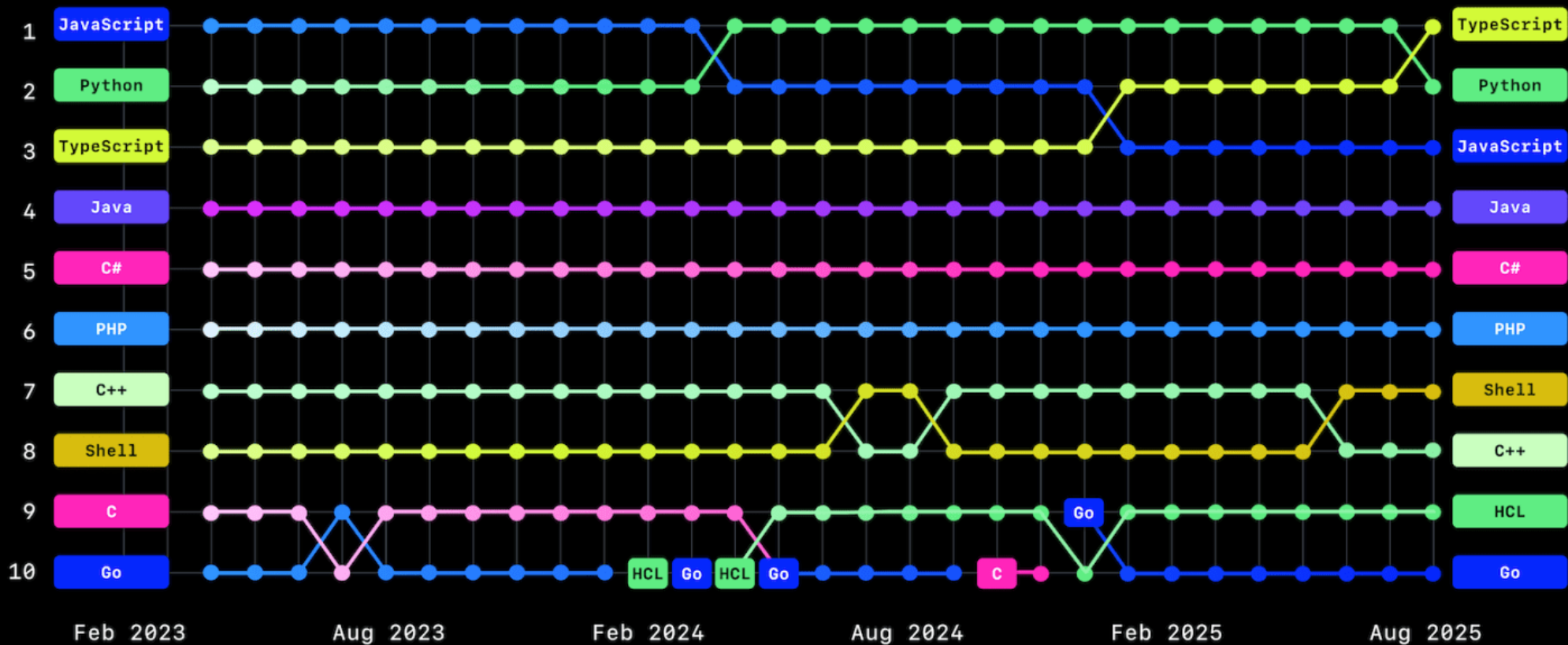
Статистика по github (2024)

# Ranking	Programming Language	Percentage (YoY Change)	YoY Trend
1	Python	18.172% (+0.129%)	
2	JavaScript	15.278% (-1.341%)	
3	Go	12.275% (+0.151%)	
4	C++	9.750% (+0.785%)	
5	Java	7.959% (-0.074%)	
6	TypeScript	7.045% (-0.298%)	
7	C	5.947% (+0.499%)	
8	C#	4.222% (+0.163%)	
9	PHP	3.690% (+0.050%)	



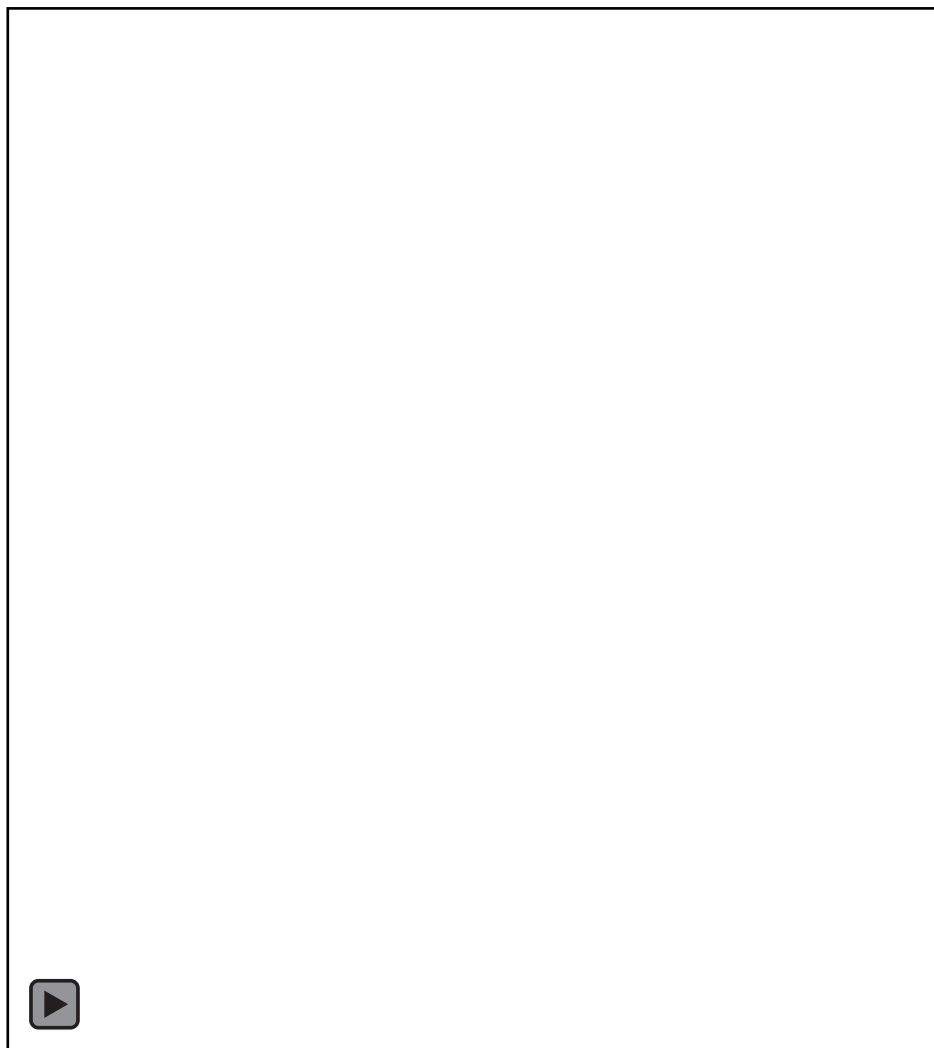
Статистика по github (2025)

Top 10 programming languages on GitHub 2023-2025





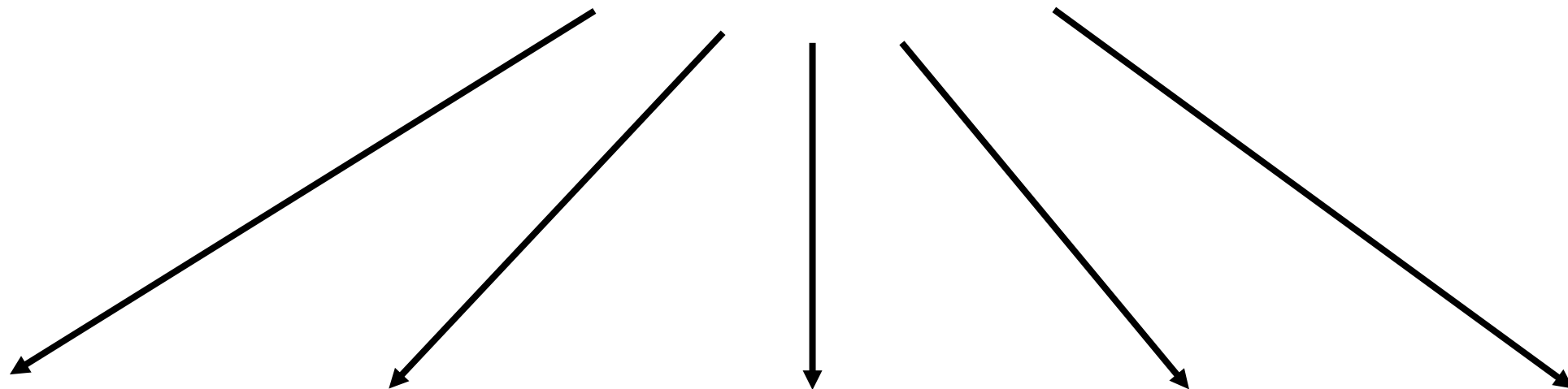
Всегда ли нужен Си?



Бонусное ДЗ:
Придумайте свой мем про
преимущества языков
C/C++. Присылайте их в ЛС.
Среди работ устроим
конкурс в чате. Лучшим
работам – бонусные баллы
по курсу. (До 5 баллов)



Области применения языков программирования



Программирование
микроконтроллеров



- C
- Assembler

Веб разработка



- PHP
- C#
- JavaScript

Приложения для
смартфонов



- Java
- Kotlin

Научные
исследования



- Python
- Matlab
- C++

Разработка
приложений



- C++
- Java
- Python

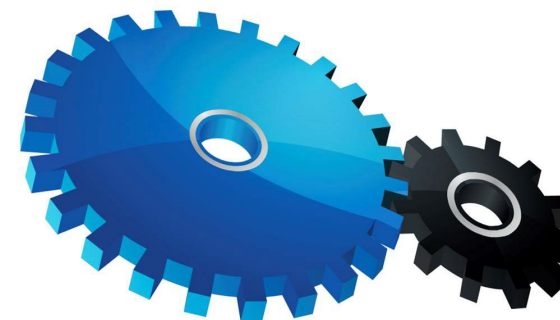
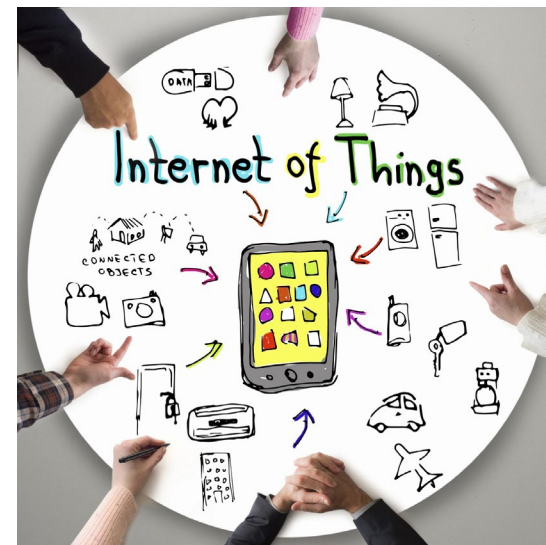


Почему мы учим именно язык «С»



Язык С применяется:

- Микроконтроллеры
- Интернет вещей
- Операционные системы
- Драйверы



Drivers



Парадигмы программирования

Парадигмы программирования

← Императивная

- Структурная
- Процедурная
- Модульная
 - ООП

→ Декларативная

- Функциональная



Парадигмы программирования

main

Настройка порта
Чтение данных
Запись данных в буфер
Чтение данных из буфера
Анализ данных
Открытие текстового файла
Запись данных
Закрытие текстового файла
Отправка сигнала завершения в
порт



Парадигмы программирования

Настройка порта

...

Чтение данных

...

Запись данных в буфер

...

Чтение данных из буфера

...

main

Отправка сигнала завершения

...

Анализ данных

...

Открытие текстового файла

...

Запись данных

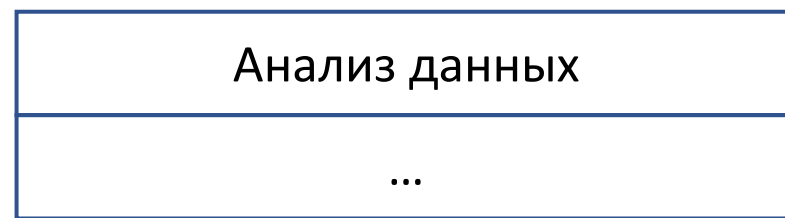
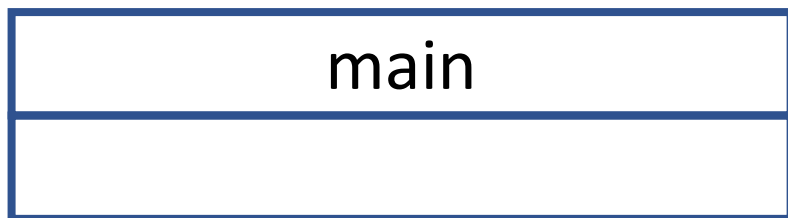
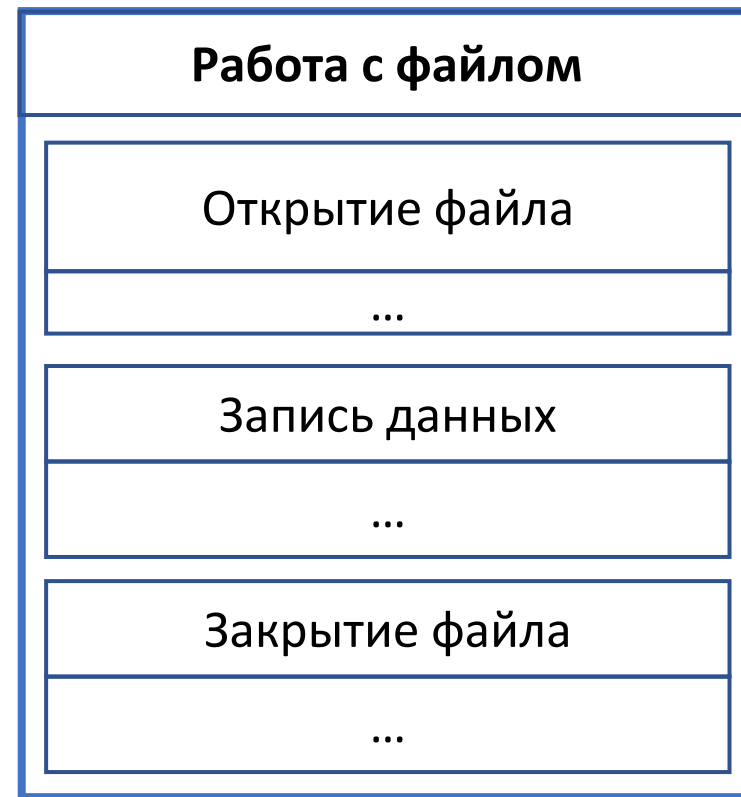
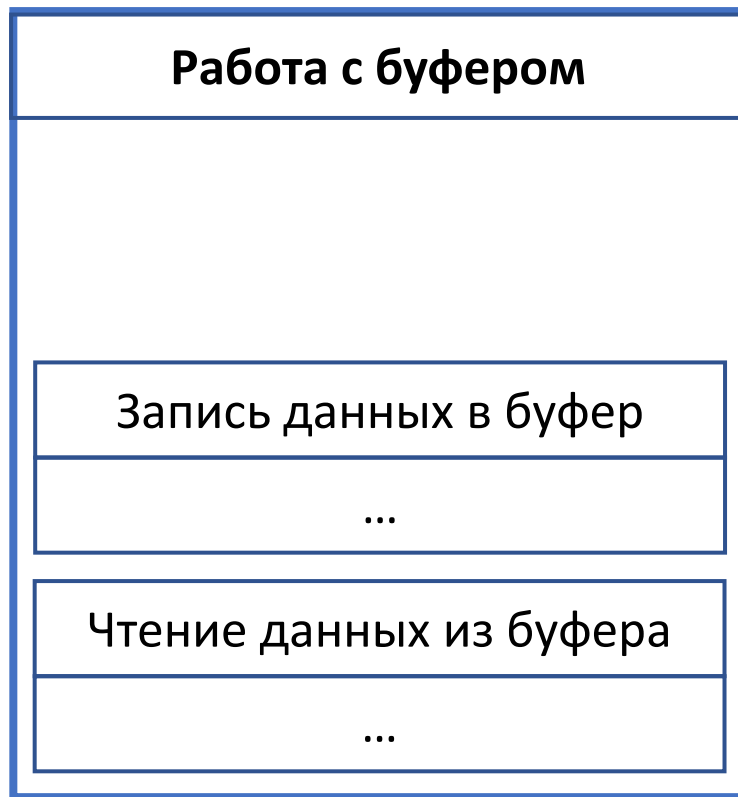
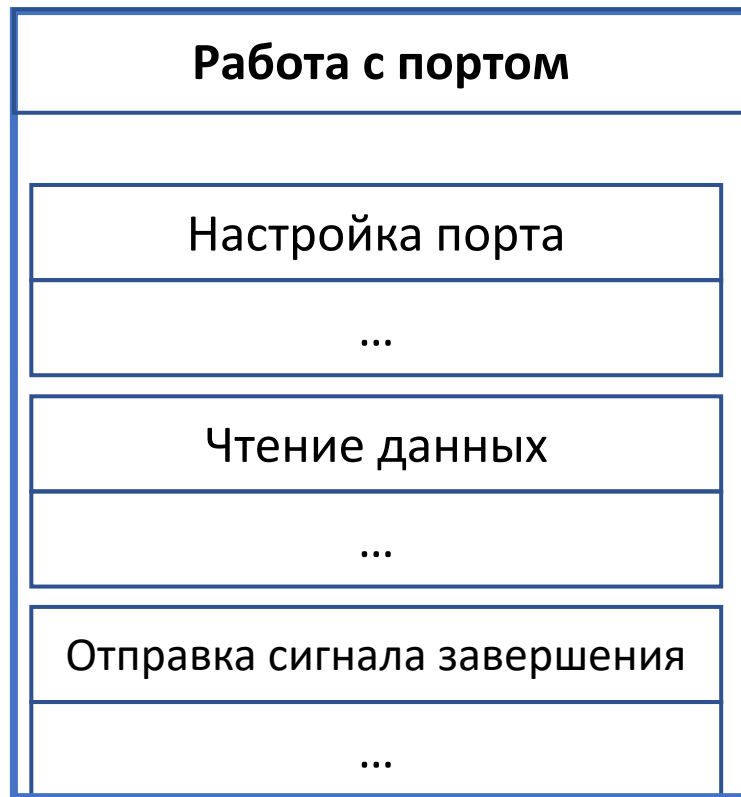
...

Закрытие текстового файла

...

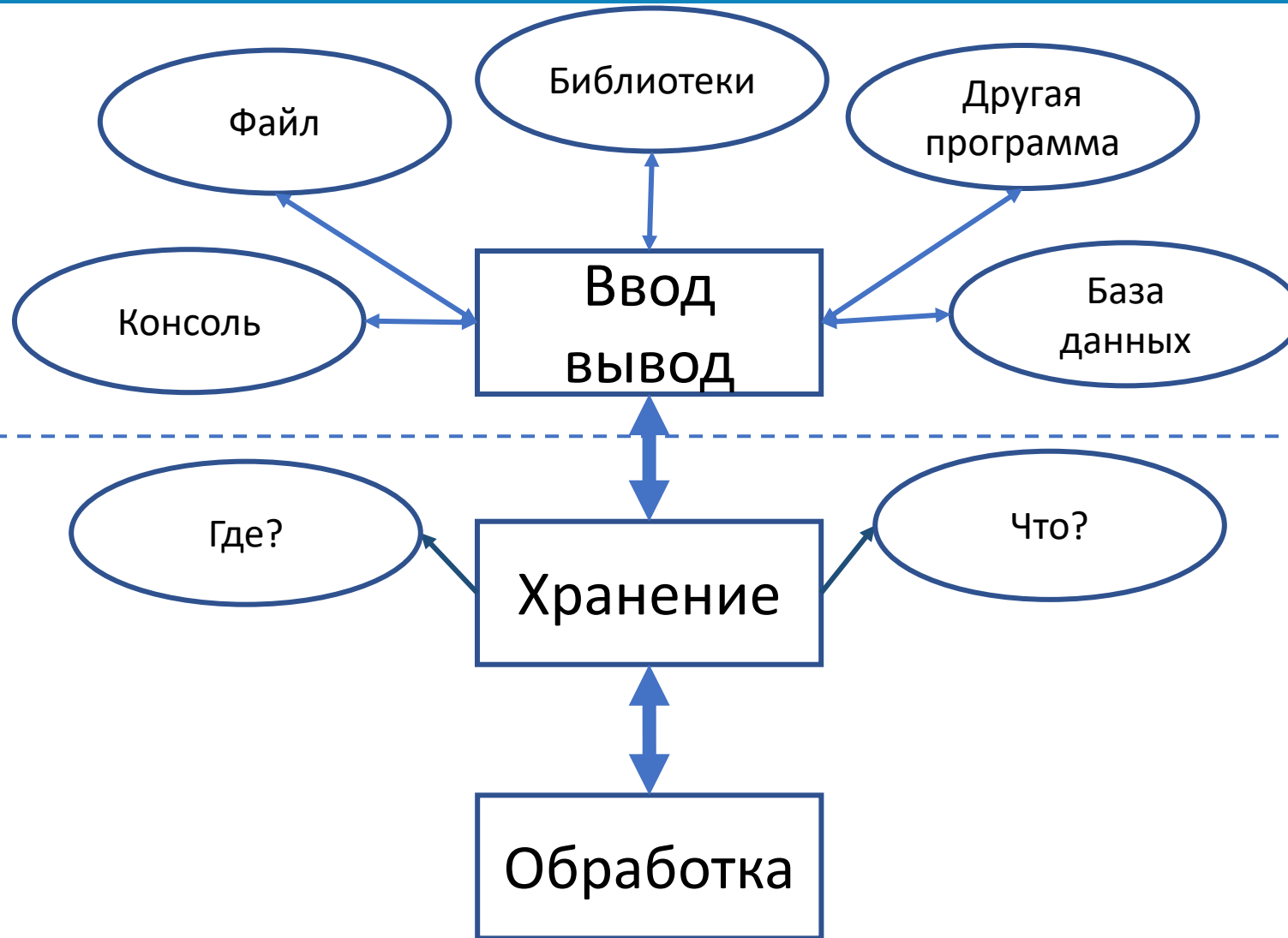


Парадигмы программирования



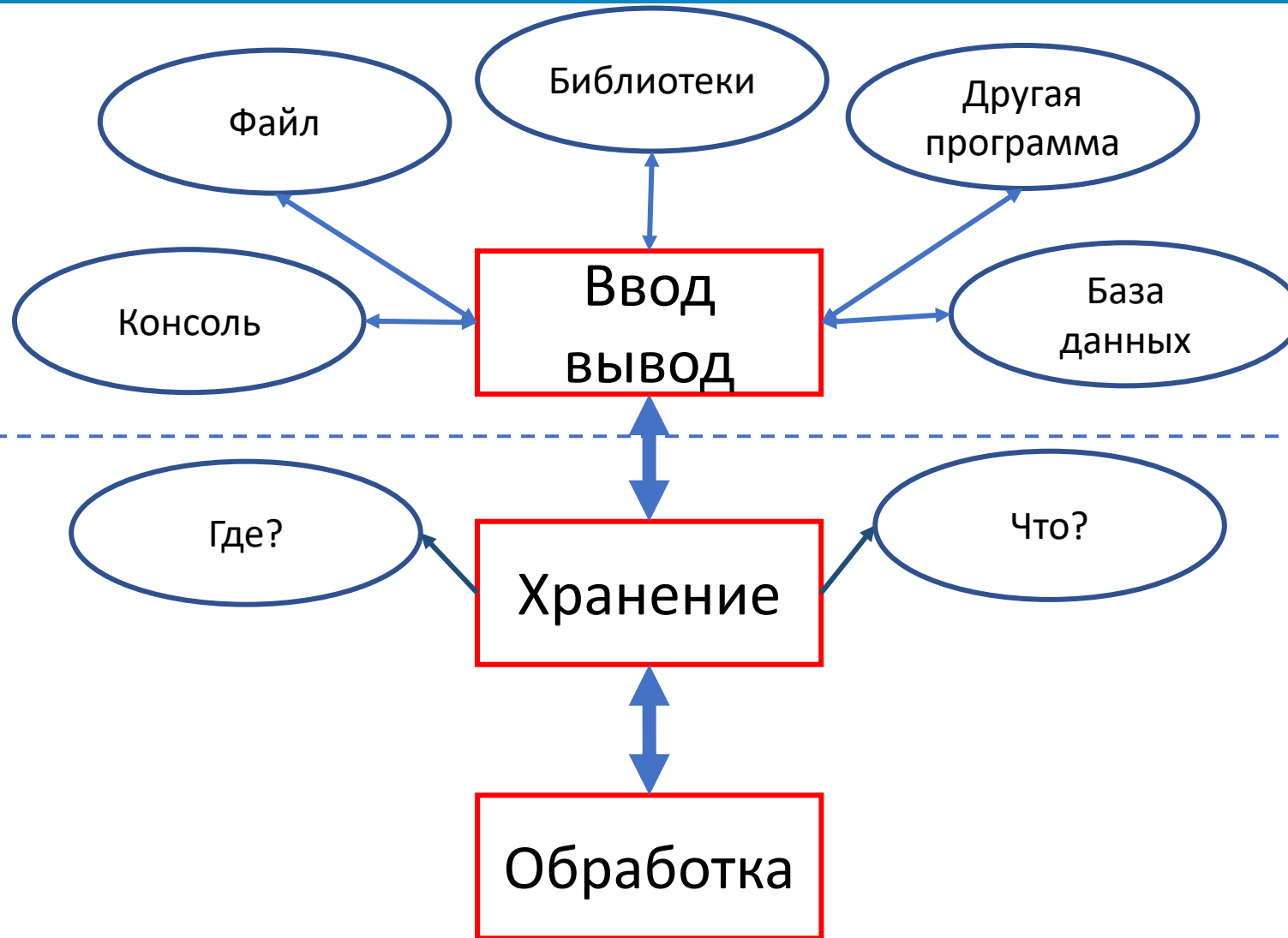


Дерево языка





Дерево языка





Подключение к WSL

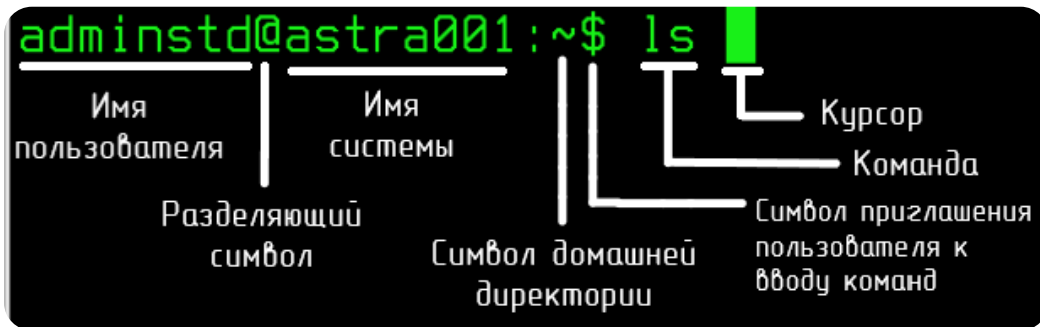
1. Создайте на диске D папку wsl
2. M: (Methodic) -> Балабаев СА -> ubuntu.vhdx копируем в созданную папку wsl (**только не все вместе!!**)
3. Запускаем скрипт wsl_import_ubuntu.ps1 (Правой кнопкой мыши -> Выполнить с помощью PowerShell -> Y)
4. В поиске находим wsl
5. Запускаем



Основа работы с командной строкой Linux



Работа с командной строкой

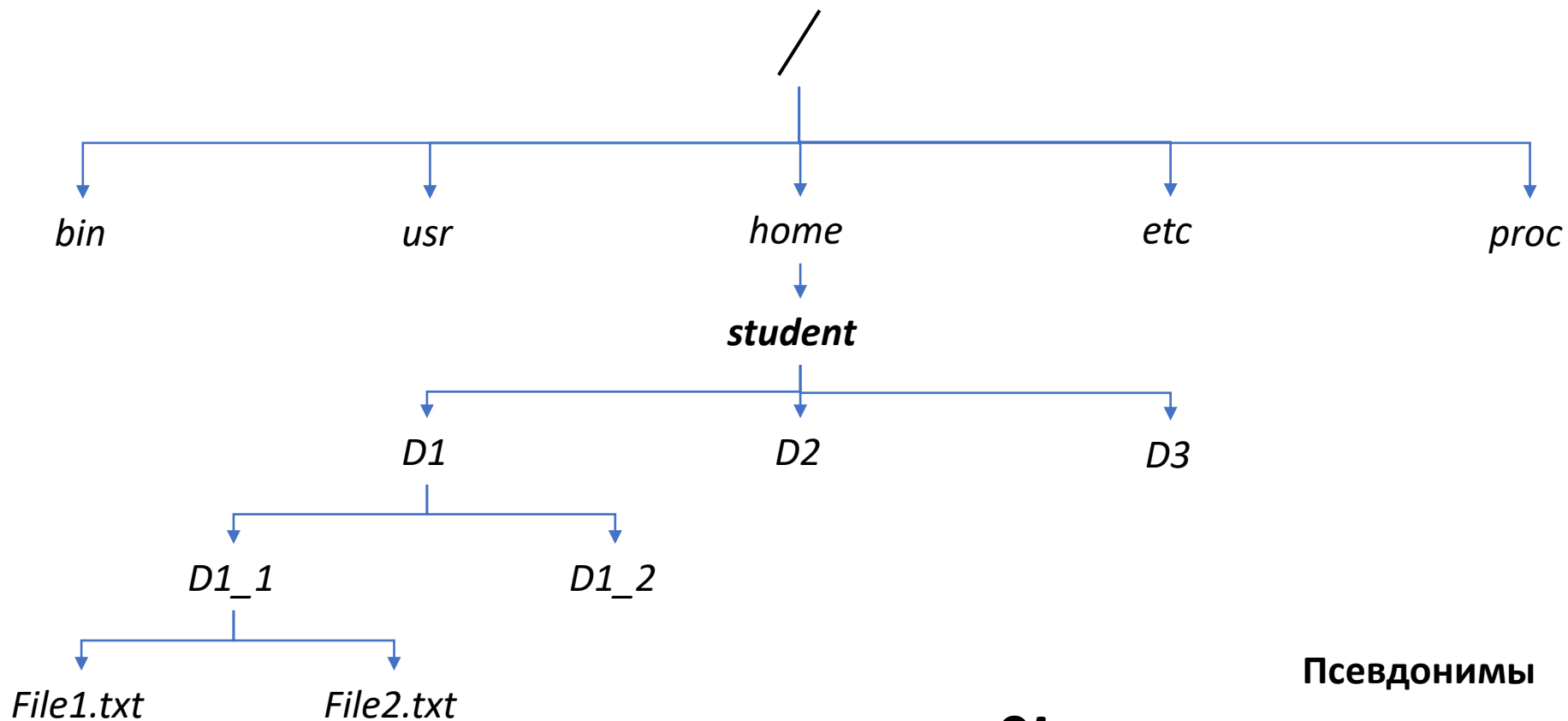


№	Команда	Описание	Пример
1	whoami	Вывести текущего пользователя Linux	whoami -> adminstd
2	pwd	Показать текущее местонахождение	pwd -> /home/user/SAB
3	ls	Позволяет просмотреть содержимое текущего каталога	ls -> dir
4	mkdir <название директории>	Создание директории	mkdir dir
5	cd <путь к директории>	Перейти в другую директорию	cd ~ /SAB/dir
6	touch <название файла>	Создание файла	touch file.txt
7	cp <что_копировать куда_копировать>	Копирование файла	cp file.txt ~/SAB/dir
8	mv <что_переместить куда_переместить>	Переместить файл	mv file.txt ~/SAB/dir
9	rm <название файла>	Удалить файл	rm file.txt
10	rm -r <название файла>	Удалить директорию	rm -r dir1
11	man	Описание работы команды	man ls
12	nano <название файла>	Редактирование файла	nano file.txt
13	echo <текст>	Печать текста	echo Hello
14	cat <название файла>	Вывести содержимое файла на экран	cat file1.txt
15	head file	Вывести первые 10 строк файла	head file.txt
16	tail file	Вывести последние 10 строк файла	tail file.txt



Директории в Linux

```
sab@SAB: ~$
```



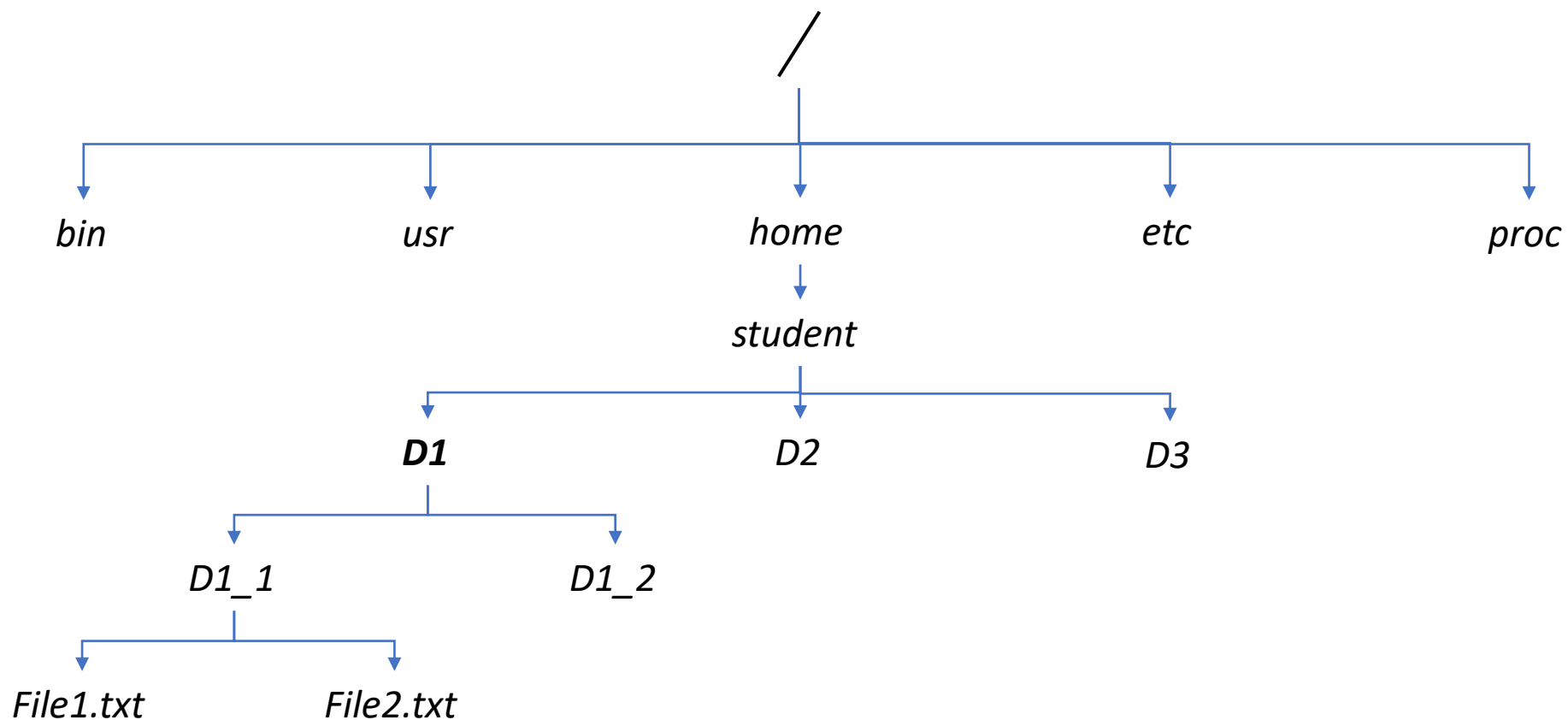
Псевдонимы

- `~` -> `/home/student` (Домашняя директория)
- `.` -> `/home/student` (Текущая директория)
- `..` -> `/home/` (Директория выше)



Директории в Linux

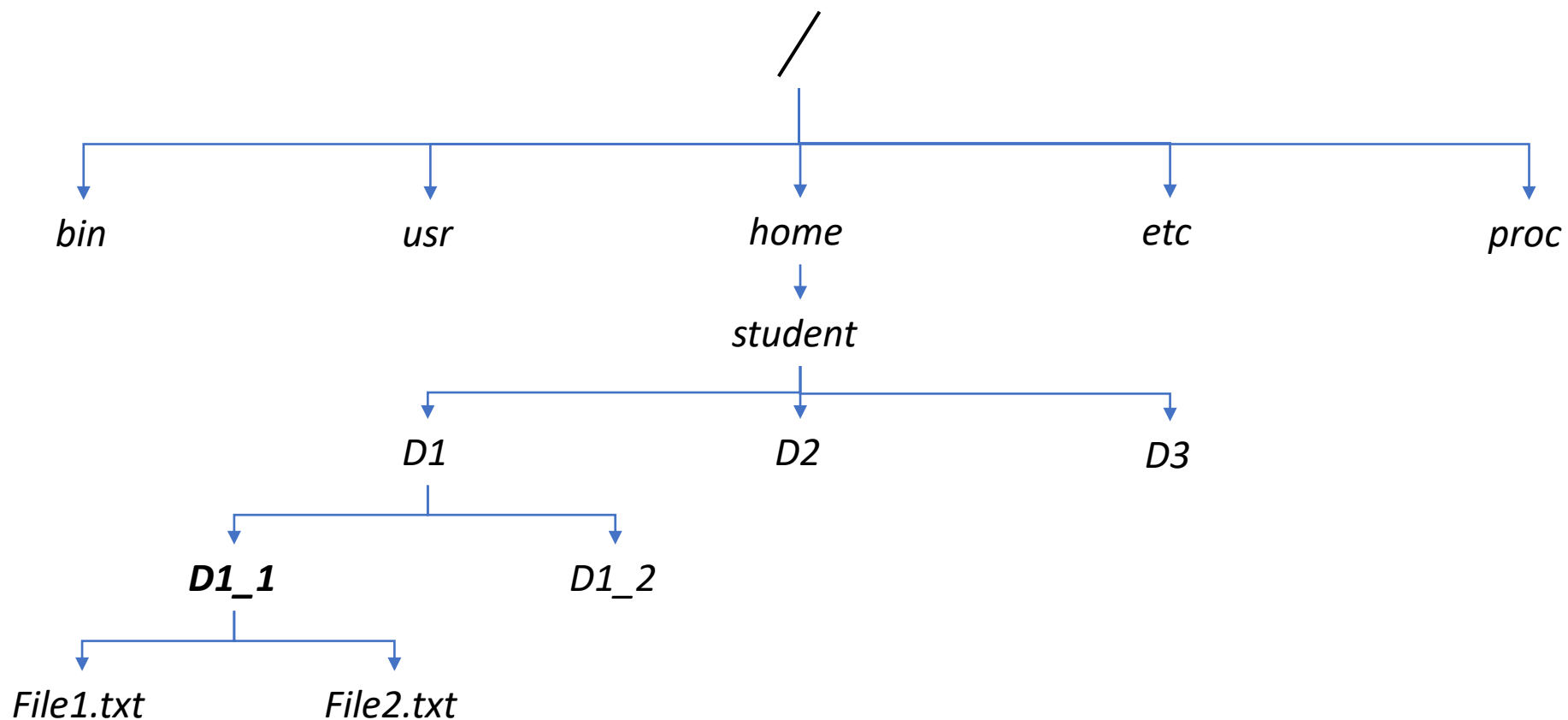
```
sab@SAB: ~ $ cd D1  
sab@SAB: ~/D1 $
```





Директории в Linux

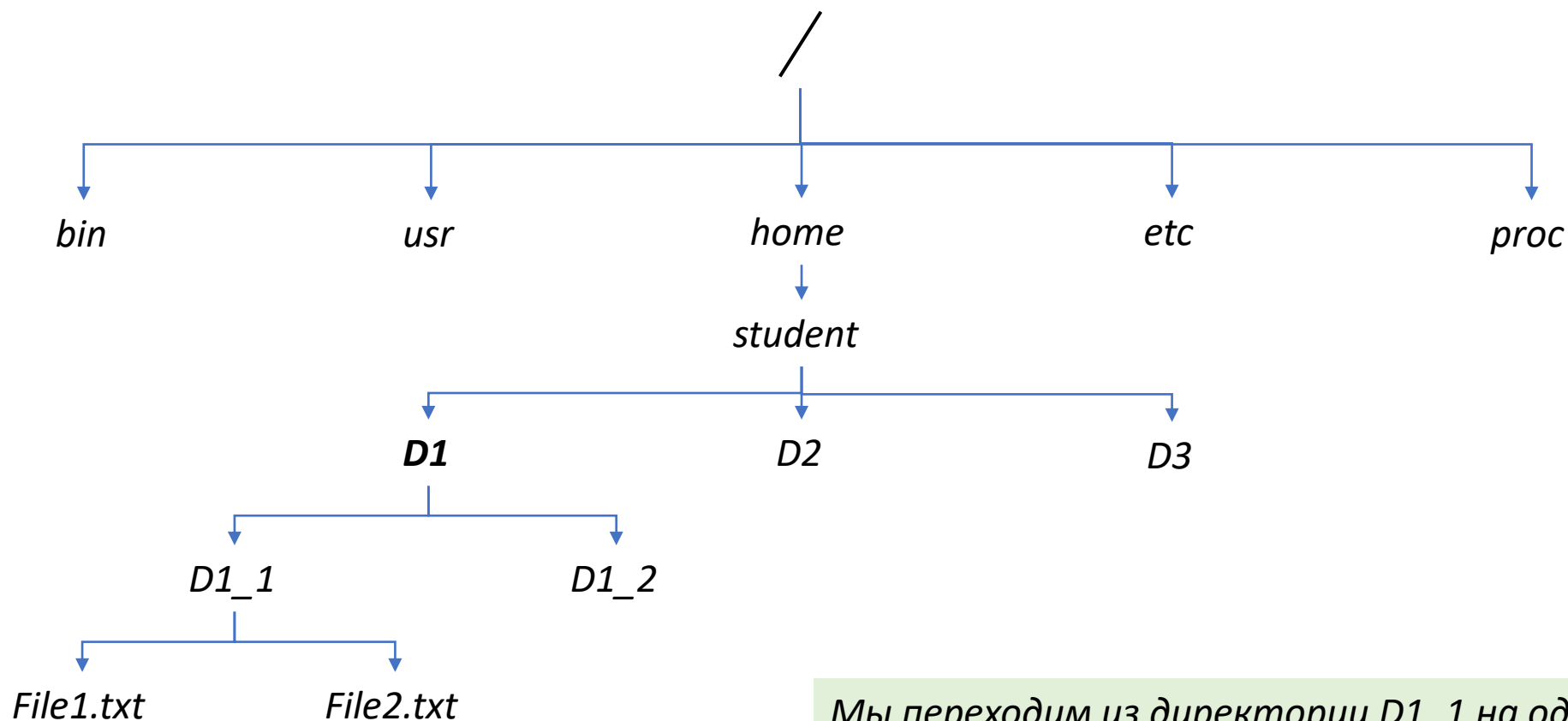
```
sab@SAB: ~/D1 $ cd D1_1  
sab@SAB: ~/D1/D1_1 $
```





Директории в Linux

```
sab@SAB: ~/D1/D1_1$ cd ..  
sab@SAB: ~/D1 $
```

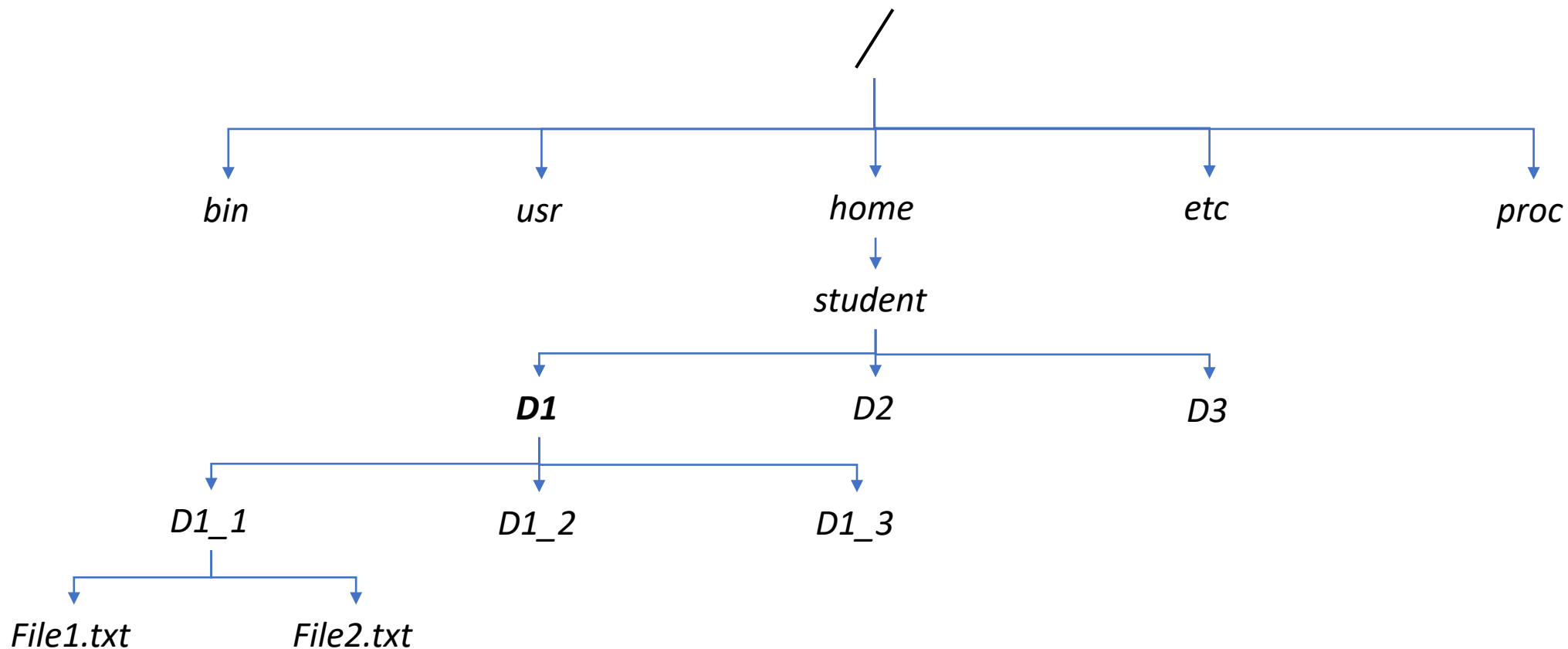


Мы переходим из директории *D1_1* на одну директорию выше используя псевдоним «..».



Директории в Linux

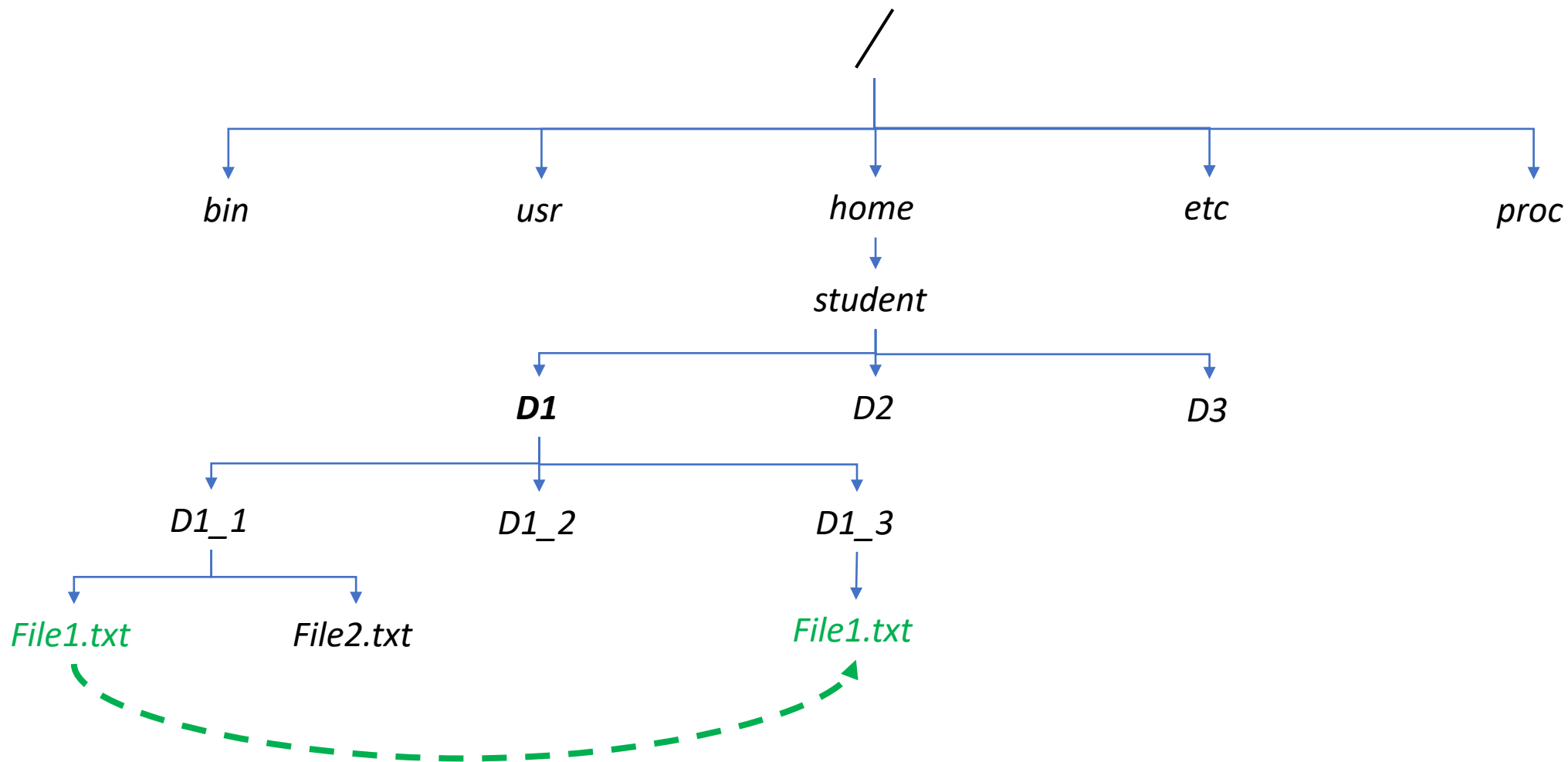
```
sab@SAB: ~/D1/$ mkdir D1_3
```





Директории в Linux

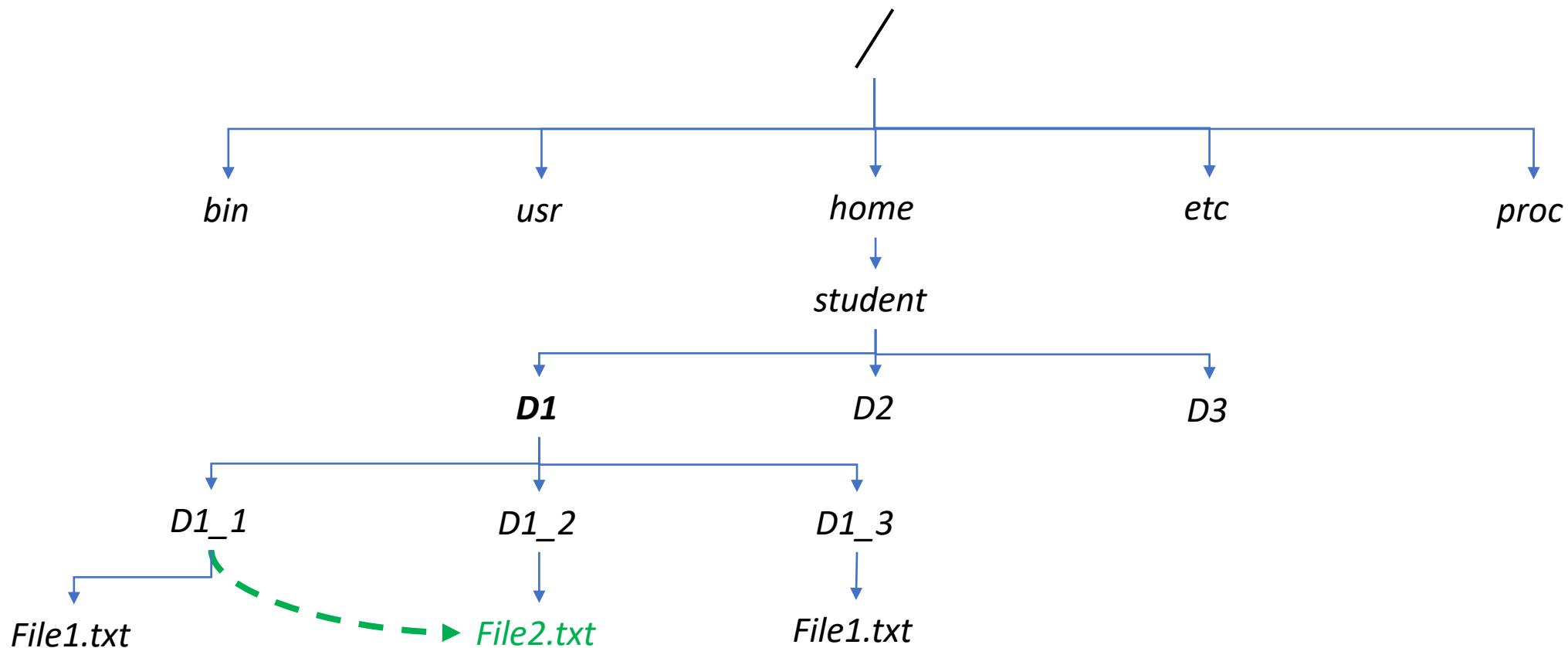
```
sab@SAB: ~/D1/$ cp D1_1/File1.txt D1_3
```





Директории в Linux

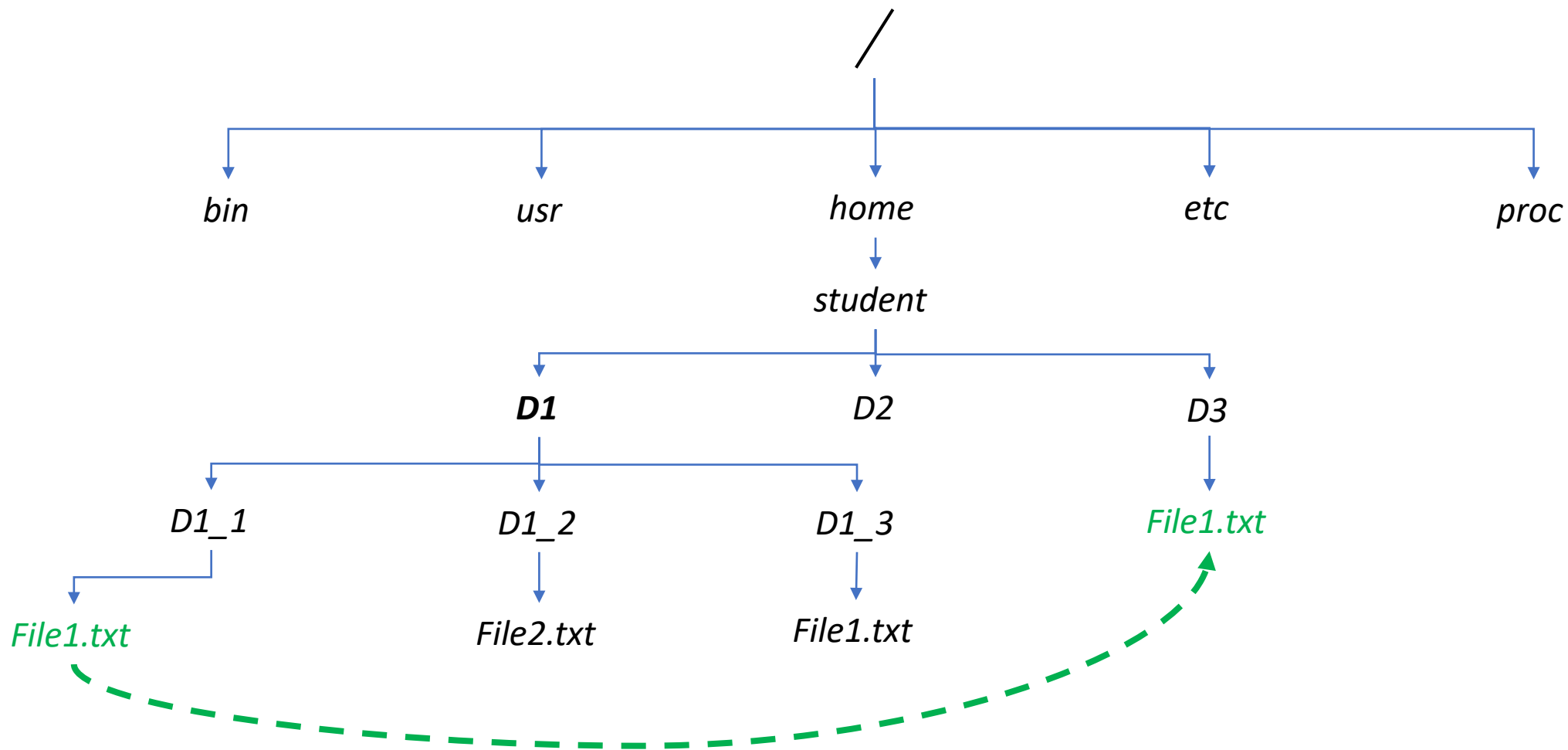
```
sab@SAB: ~/D1/$ mv D1_1/File2.txt D1_2
```





Директории в Linux

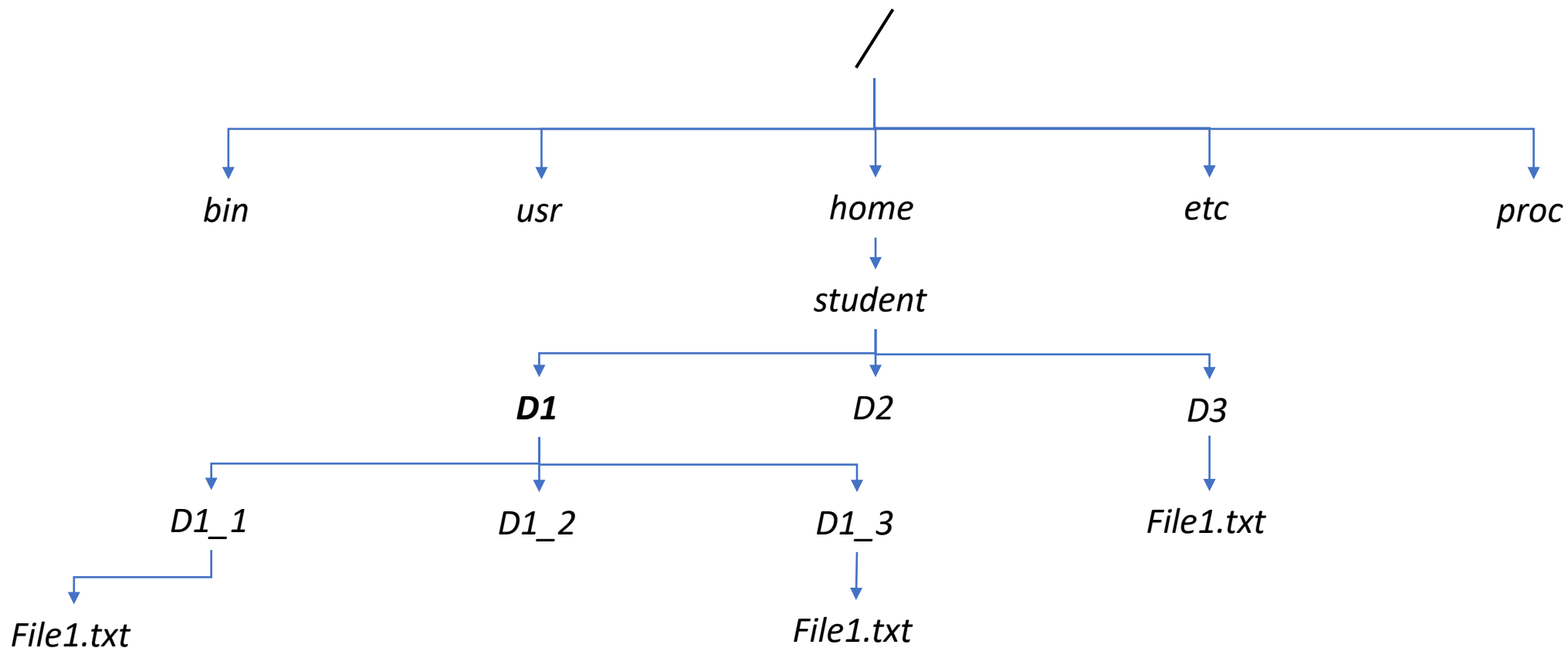
```
sab@SAB: ~/D1/$ cp D1_1/File1.txt ../D3
```





Директории в Linux

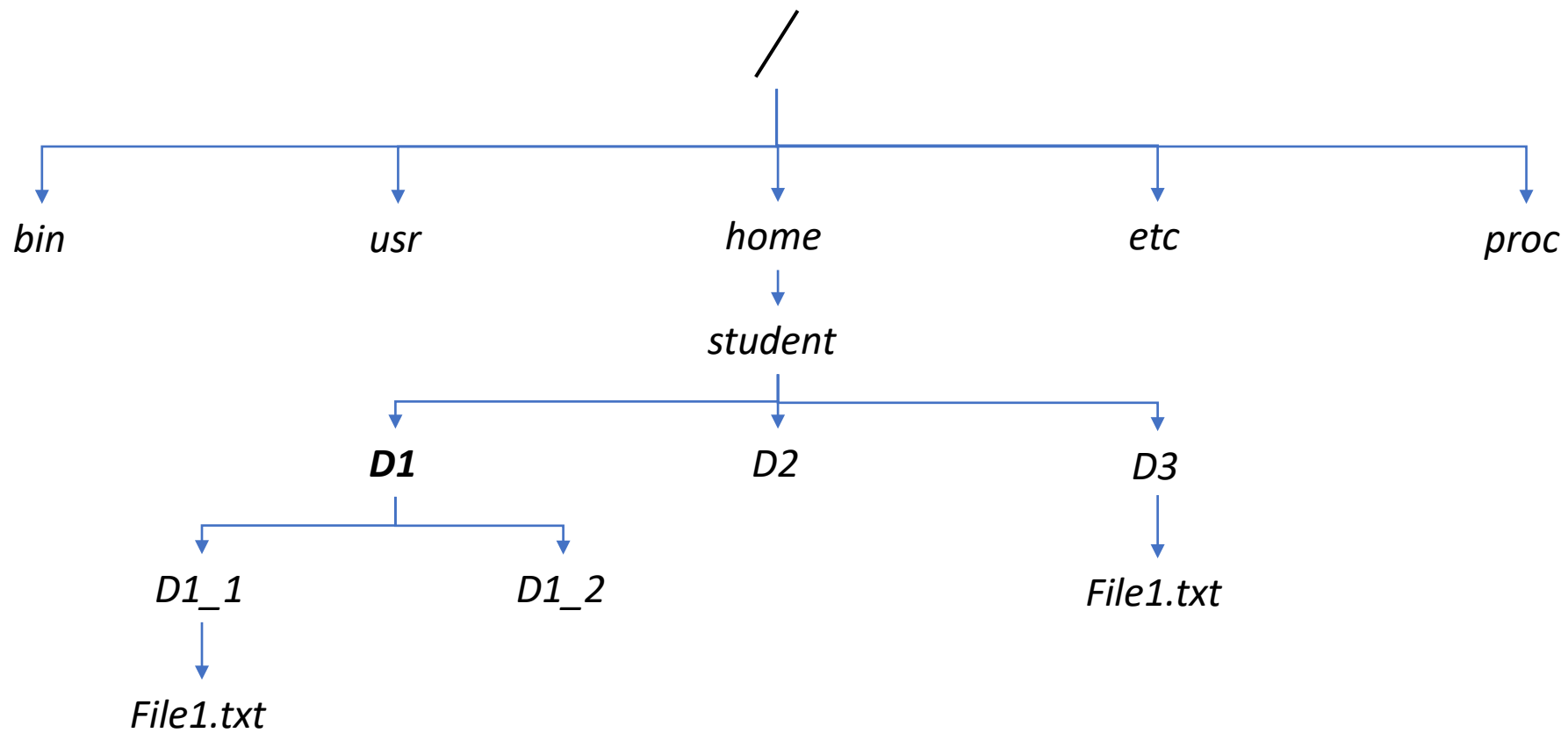
```
sab@SAB: ~/D1/$ rm D1_2/File2.txt
```





Директории в Linux

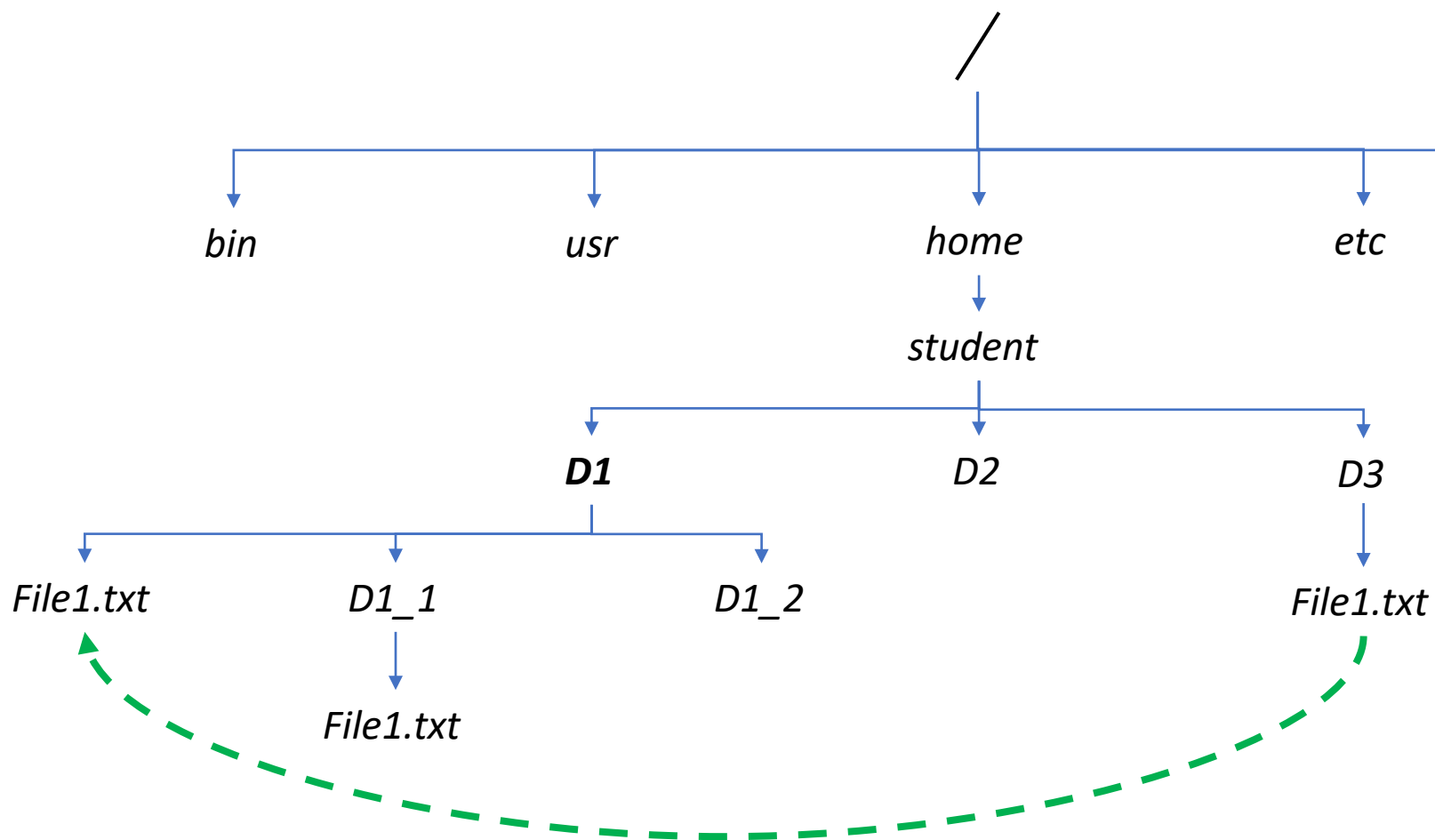
```
sab@SAB: ~/D1/$ rm -r D1_3
```





Директории в Linux

```
sab@SAB: ~/D1/$ cp ../D3/File1.txt .
```



Мы копируем из директории *D3* в текущую директорию. Для этого мы с помощью «*..*» обращаемся к директории выше, из неё заходим в *D3*, находим файл и копируем в текущую директорию «*.*»



Логические операции объединения команд

Символ	Комментарий
A ; B	Оператор “;” выполняет все команды независимо от того, завершились ли предыдущие неудачно или нет
A && B	Оператор “&&” или И выполняет вторую команду только в том случае, если предыдущая команда выполнена успешно
A B	Оператор “ ” ИЛИ выполняет вторую команду только в том случае, если предыдущая команда возвращает ошибку.

```
sab@SAB: /$ cat file ; echo I will always work
```

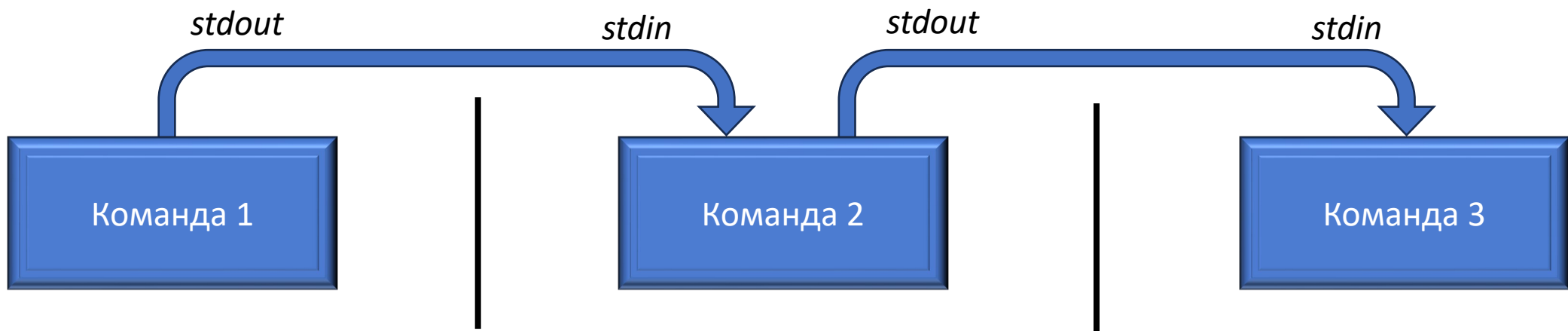
```
sab@SAB: /$ cat file && echo I will work only when file exists
```

```
sab@SAB: /$ cat file || echo I will work only when file NOT exists
```



Pipes (Каналы)

Каналы используются для перенаправления потока из одной программы в другую.



```
sab@SAB: /$ ps | grep p
```

```
sab@SAB: /$ ps > 1.txt; ls >> 1.txt; cat 1.txt | grep a
```



Символы подстановки

Символы подстановки используются для составления шаблонов для имен файлов и позволяют работать с группой файлов:

Метасимвол	Описание работы
*	заменяет любое количество любых символов
?	заменяет любой одиночный (непустой) символ
[диапазон_символов]	заменяет одиночный символ из диапазона
[!диапазон_символов]	заменяет любой символ из НЕ диапазона

Вывод информации о всех файлах название которых начинается со слова file

```
sab@SAB: /$ ls file*
```

Вывод информации о всех файлах название которых состоит из символов и начинается со слова file

```
sab@SAB: /$ ls file?
```



Несколько полезных команд Linux

hexdump — показывает шестнадцатеричное представление данных, поступающих на стандартный поток ввода.

cat — считывает данные со стандартного потока ввода и передает их на стандартный поток вывода.

sudo - запуск программы от имени других пользователей, а также от имени суперпользователя.

bc — калькулятор

python3 — среда разработки Python

head — чтение первых 10 строк файла

tail — чтение последних 10 строк файла

alias — создание псевдонимов для команд

```
sab@SAB: /$ echo "10*10" | bc
```

```
sab@SAB: /$ alias sn="sudo nano"
```




Перенаправление ввода и вывода





Перенаправление ввода и вывода

Ввод и вывод распределяется между тремя стандартными потоками:

- **stdin** — стандартный ввод (клавиатура), - 0
- **stdout** — стандартный вывод (экран), - 1
- **stderr** — стандартная ошибка (вывод ошибок на экран). - 2

< file — использовать файл как источник данных для стандартного потока ввода.

> file — направить стандартный поток вывода в файл (перезапись)

2> file — направить стандартный поток ошибок в файл (перезапись)

>>file — направить стандартный поток вывода в файл (добавление)

2>>file — направить стандартный поток ошибок в файл. (добавление)

&>file или **>&file** — направить с.п. вывода и с.п. ошибок в файл.



Перенаправление ввода и вывода

```
sab@SAB: /$ ps > 1.txt
sab@SAB: /$ cat 1.txt
PID TTY      TIME CMD
 480 pts/2    00:00:00 bash
 598 pts/2    00:00:00 ps
sab@SAB: /$ ps >> 1.txt
sab@SAB: /$ cat 1.txt
PID TTY      TIME CMD
 480 pts/2    00:00:00 bash
 598 pts/2    00:00:00 ps
PID TTY      TIME CMD
 480 pts/2    00:00:00 bash
 600 pts/2    00:00:00 ps
```

```
sab@SAB: /$ ps qq > 1.txt
error: process ID list syntax error
///
sab@SAB: /$ ps qq 2>/dev/null
sab@SAB: /$
```

Команда указана с ошибкой. На экране сообщение об ошибке
При перенаправлении потока ошибок в /dev/null – на экране пусто

```
#!/usr/bin/python3
x = input()
print(x)
prent(x)
```

scr.py



```
sab@SAB: /$ ./scr.py > output < input 2> error
sab@SAB: /$
```

```
input -> hello
output -> hello
error -> ... NameError: name 'prent' is not defined
```

После выполнения скрипта на экране – пусто. Ввод считался с файла input (там была строка hello). Вывод произошел в файл output (строка hello). Сообщение об ошибке сохранилось в файл error



Основы работы с командной строкой Linux

```
sab@SAB:.../Lesson1$
```

nano Program.c

```
#include <stdio.h>
int main()
{
    printf("Hello World!\n");
    return 0;
}
```

gcc Program.c -o Program

```
sab@SAB:.../Lesson1$ ./Program
Hello World!
```



Потренируемся

1. Откройте терминал *Linux*
2. Создайте директорию. Назовите ее по вашему номеру студенческого билета
3. Войдите внутрь директории
4. Создайте внутри нее директорию *Lab1*
5. Не входя внутрь *Lab1* создайте в нем файл с названием README.md
6. Создайте в директории файл *lab1.txt*, который содержит ваши ФИО и номер группы
7. Перейдите в директорию *Lab1*
8. Скопируйте файл *lab1.txt* в директорию *Lab1*
9. Перенесите содержимое файла *lab1.txt* в файл README.md.
10. Удалите файл *lab1.txt*
11. Используя текстовый редактор добавьте в файл README.md строку с текстом «Лаба готова»